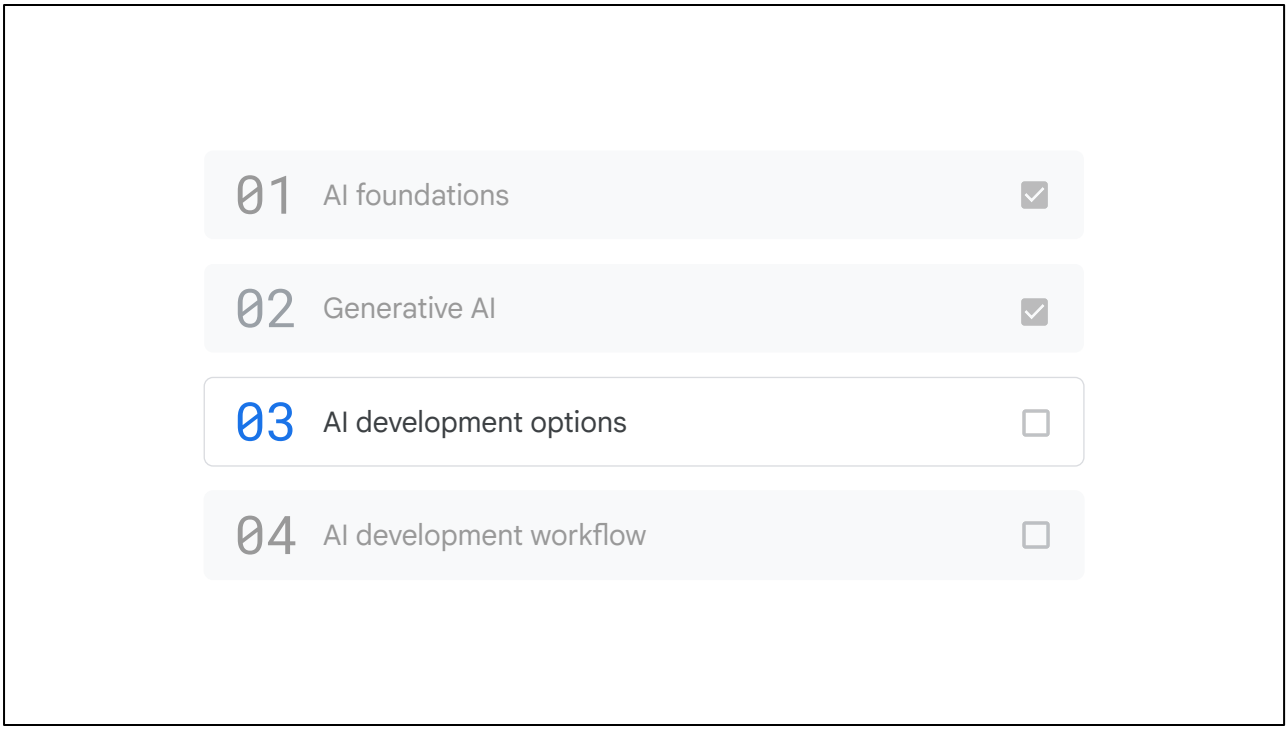
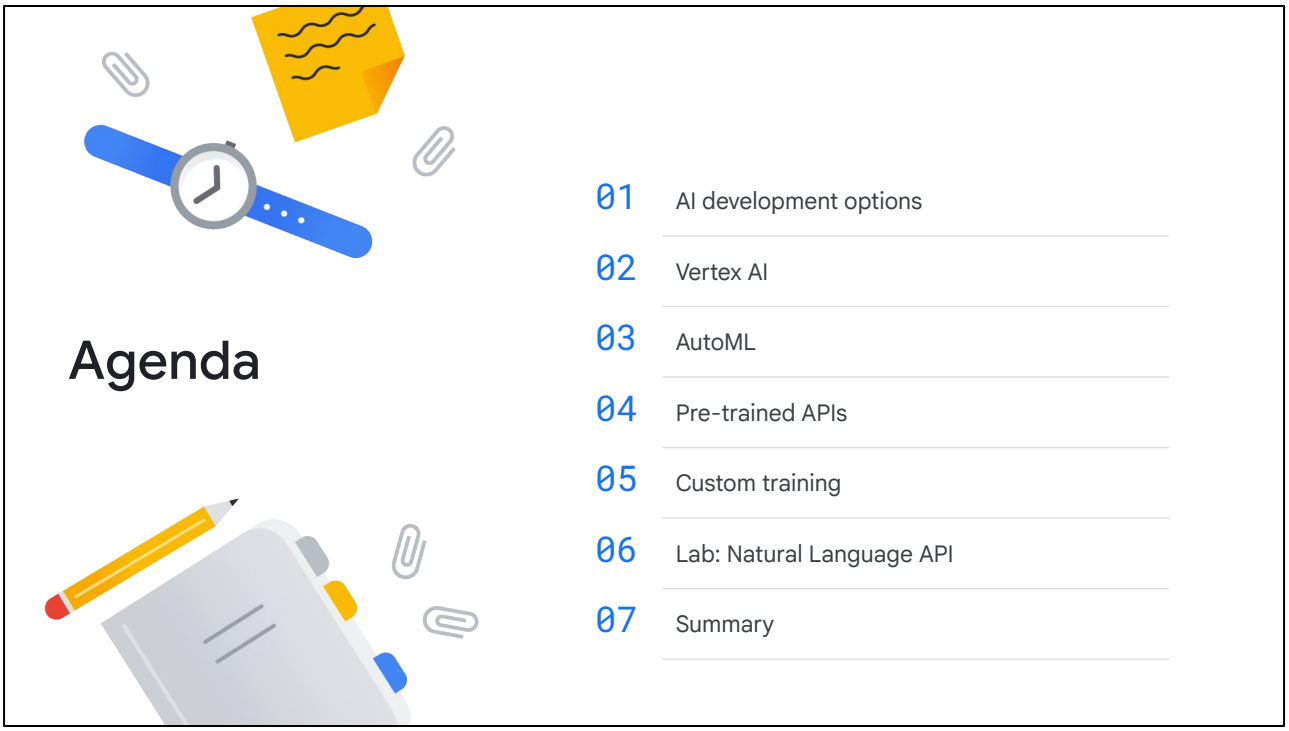


In the previous module, you explored cutting-edge technologies about generative AI. But what if you want to create predictive AI for traditional AI tasks like forecasting and classification?

How do you build an ML model, and what options do you have? Find the answers in this module.

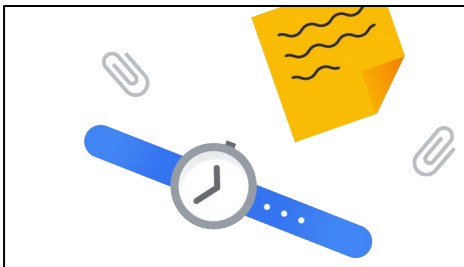


You begin by comparing AI development options on Google Cloud from no-code to low-code. And finally, a do-it-yourself approach. These options apply to both gen AI and predictive AI.

You are then introduced to Vertex AI, Google's unified AI development platform and your playground to build an ML model from end to end.

After that, you take a deep dive into how each of three options works: AutoML, pre-trained APIs, and custom training.

Never miss a practice! You conclude with hands-on experience using the Natural Language API to identify subjects and analyze sentiment in text.



Agenda

01 AI development options

02 Vertex AI

03 AutoML

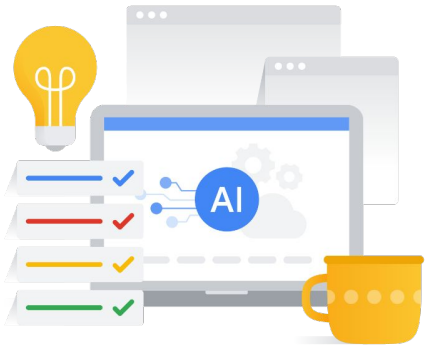
04 Pre-trained APIs

05 Custom training

06 Lab: Natural Language API

07 Summary

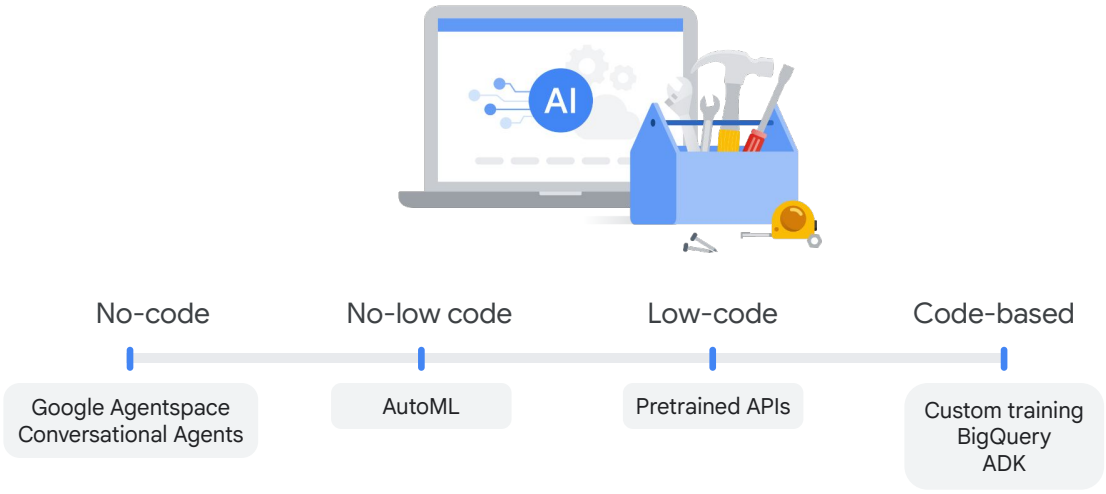
Let's start with Google Cloud's AI development options. What choices do you have for both generative AI and predictive AI projects? And how do you make the right decision? Let's find out!



- Business users
- Data scientists
- ML engineers

Imagine transforming your organization's business model and operations with AI.

- Perhaps you're a business user without a technical background, but you're eager to prototype business ideas using AI. What are your options?
- Or maybe you're a data scientist with training data, looking to build a custom ML model without spending hours tuning parameters from scratch. What choices do you have?
- Even if you're an ML engineer who enjoys a do-it-yourself approach to building ML pipelines, what tools can you utilize?



Google Cloud can help you achieve your goals by meeting you where you are, offering:

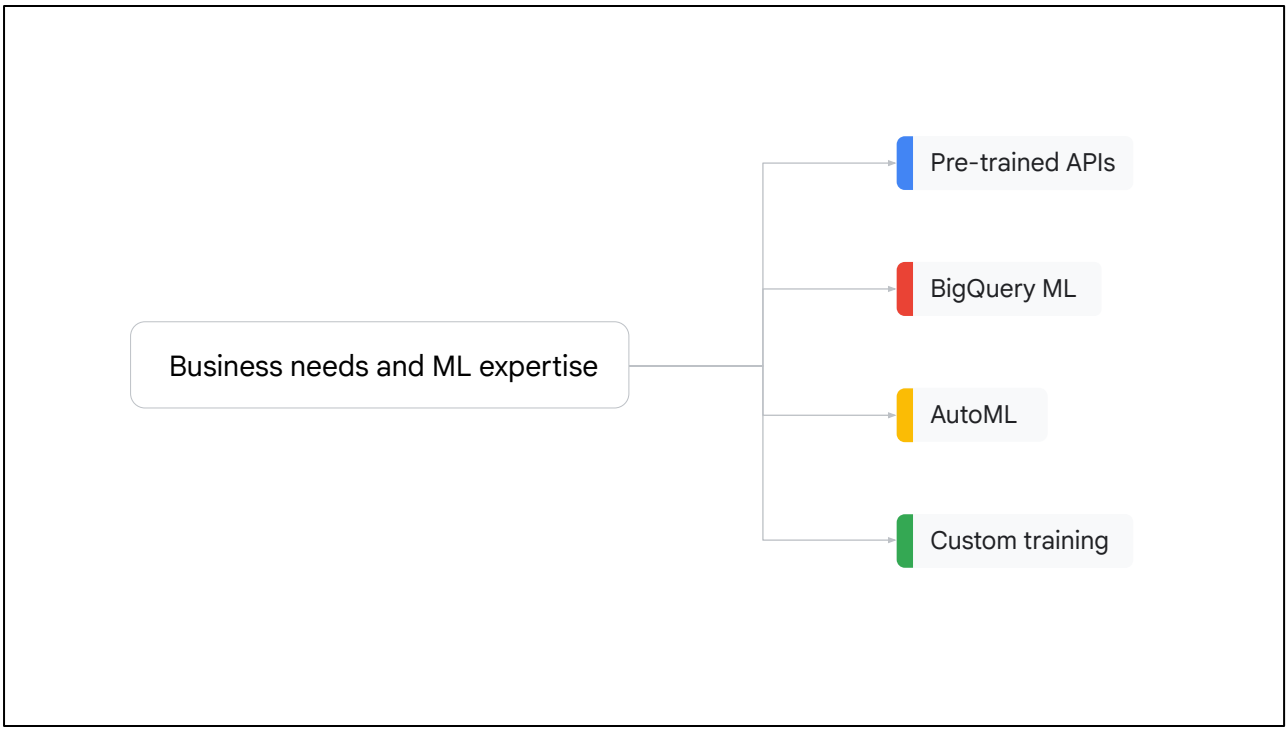
- **No-code** out-of-box solutions like Google Agentspace and Conversational Agents, introduced in the previous Gen AI module.
- **No- to low-code** solutions like AutoML, which helps you build your own ML models through point-and-click.
- **Low-code** solutions like preconfigured APIs, which use pre-trained ML models, eliminating the need to build your own if you lack training data or in-house ML expertise.
- **Code-based** approaches, ranging from BigQuery ML and ADK (Agent Development Kit), introduced earlier, to completely DIY custom training.

	Pre-trained APIs	BigQuery ML	AutoML	Custom training
Data type	Tabular, image, text, audio, and video	Tabular	Tabular, image, text, and video	Tabular, image, text, and video
Training data size	No data required	Medium to large	Small to medium	Medium to large
ML and coding expertise	Low	Medium	Low	High
Flexibility to tune hyperparameters	None	Medium	None	High
Time to train a model	None	Medium	Medium	Long

Beyond technical expertise, how do you choose the right tool to build an ML model?

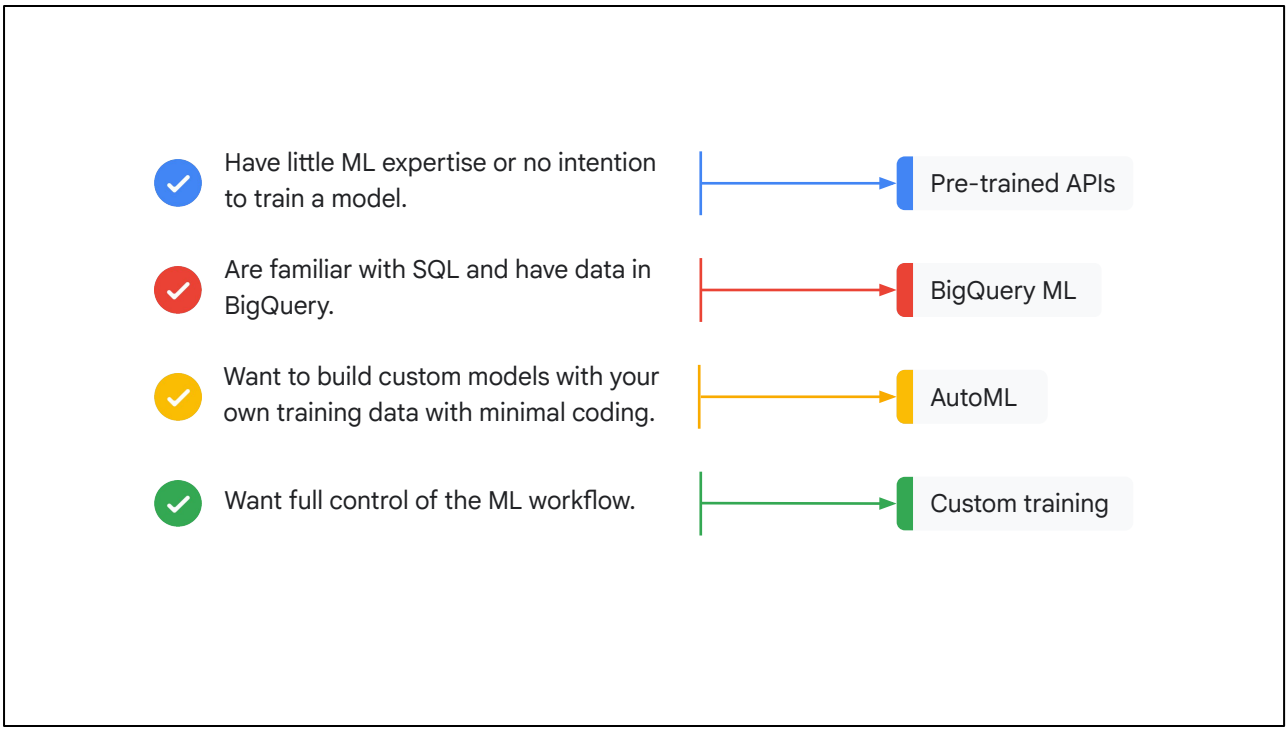
This brief guide comparing four options—pre-trained APIs, BigQuery ML, AutoML, and custom training—may offer some insight:

- BigQuery ML only supports tabular data and semi-structured data like JSON file, whereas the other three support tabular, image, text, and video. Pre-trained APIs also process audio.
- In terms of training data size, pre-trained APIs do not require any training data, whereas BigQuery ML and custom training require a large amount of data.
- Pre-trained APIs and AutoML are user friendly with low requirements for machine learning and coding expertise, whereas custom training has the highest requirement, and BigQuery ML requires you to understand SQL.
- At the moment, you can't tune the hyperparameters with pre-trained APIs or AutoML. However, you can experiment with hyperparameters by using BigQuery ML and custom training.
- Pre-trained APIs require no time to train a model because they directly use pre-trained models from Google. The time to train a model for the other three options depends on the specific project. Normally, custom training takes the longest time, because it builds the ML model from the beginning, unlike AutoML and BigQuery ML.



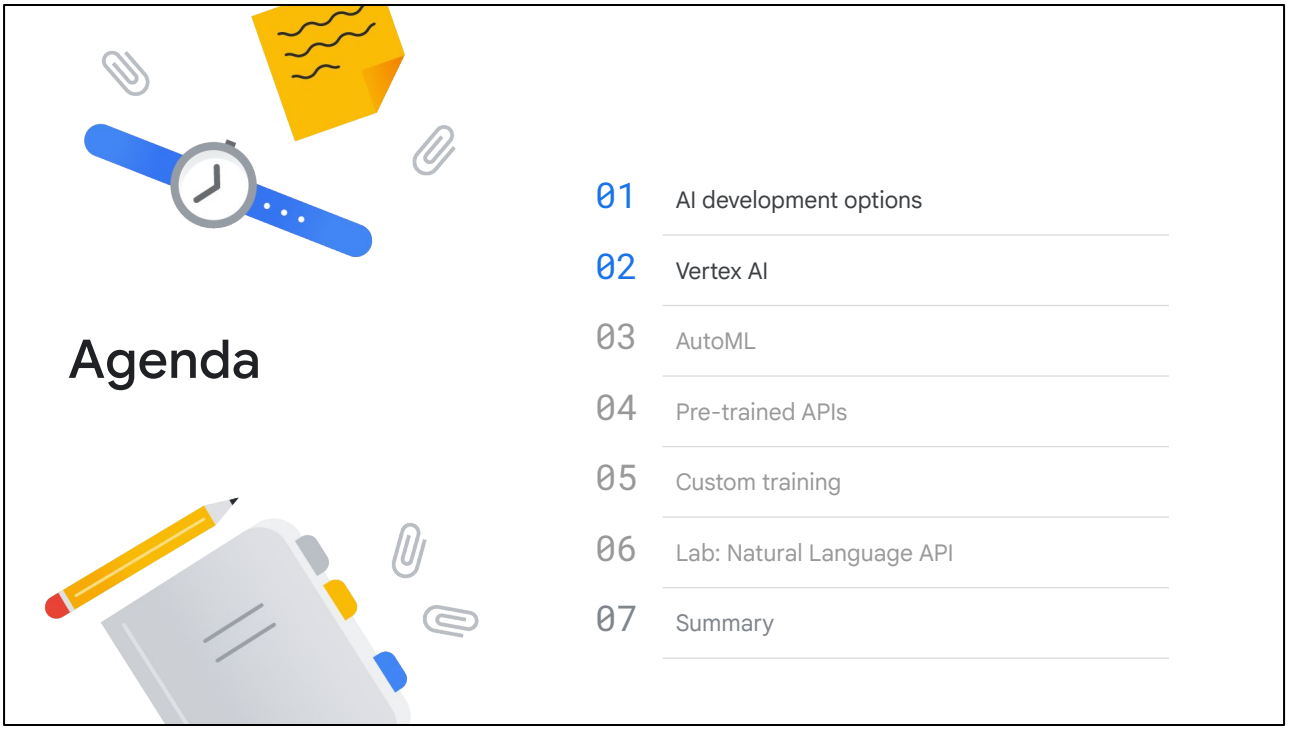
The best option depends on your business needs and ML expertise.

Budget is also an important consideration. Visit Google Cloud's website for detailed pricing information.

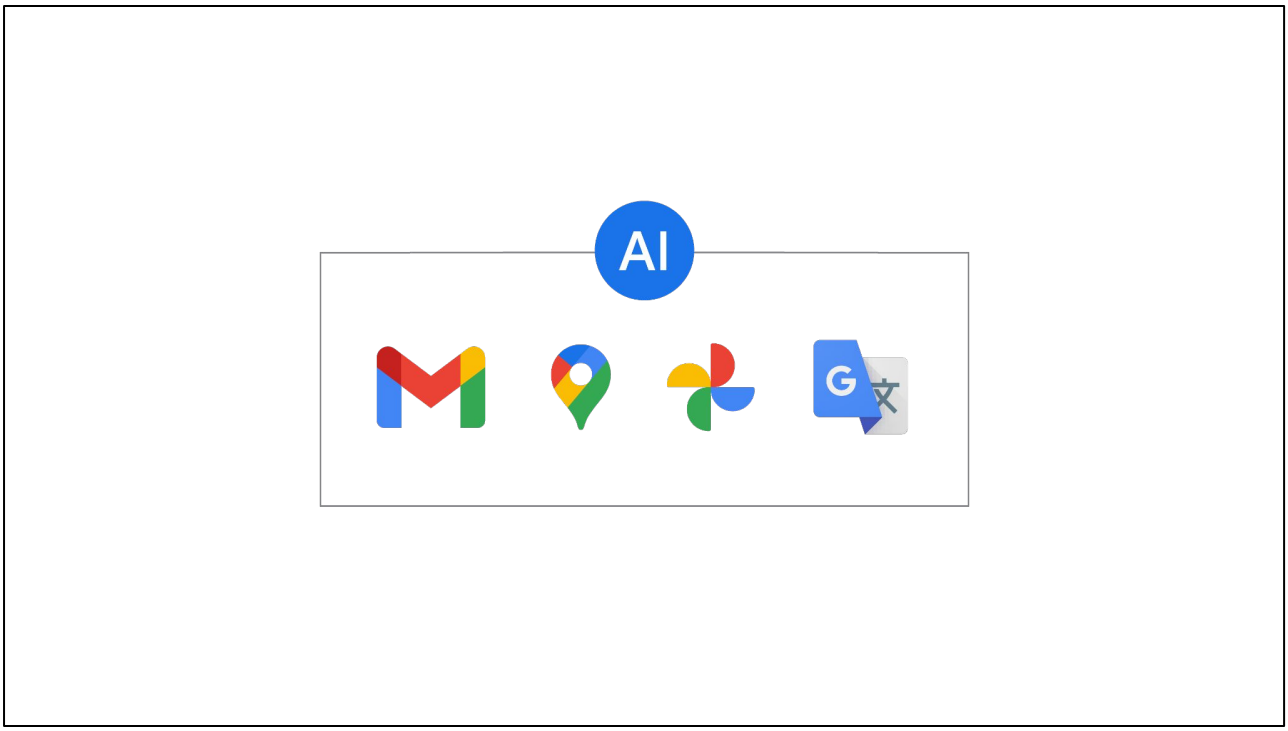


- If you have little ML experience and no intention to train your own ML models, using pre-trained APIs might be the best choice. Pre-trained APIs address common perceptual tasks such as vision, video, and natural language. They are ready to use without any model development effort.
- If your data engineers, scientists, or analysts are familiar with SQL and already have data in BigQuery, BigQuery ML lets you use SQL queries to build predefined ML models.
- If you wish to build custom models with your own training data while you spend minimal time coding, then AutoML on Vertex AI is your choice. AutoML allows you focus on business problems instead of the underlying model architecture and provisioning.
- If your ML engineers and data scientists want full control of the ML workflow, Vertex AI custom training lets you train and serve custom models with code on Vertex AI Workbench or Colab Enterprise.

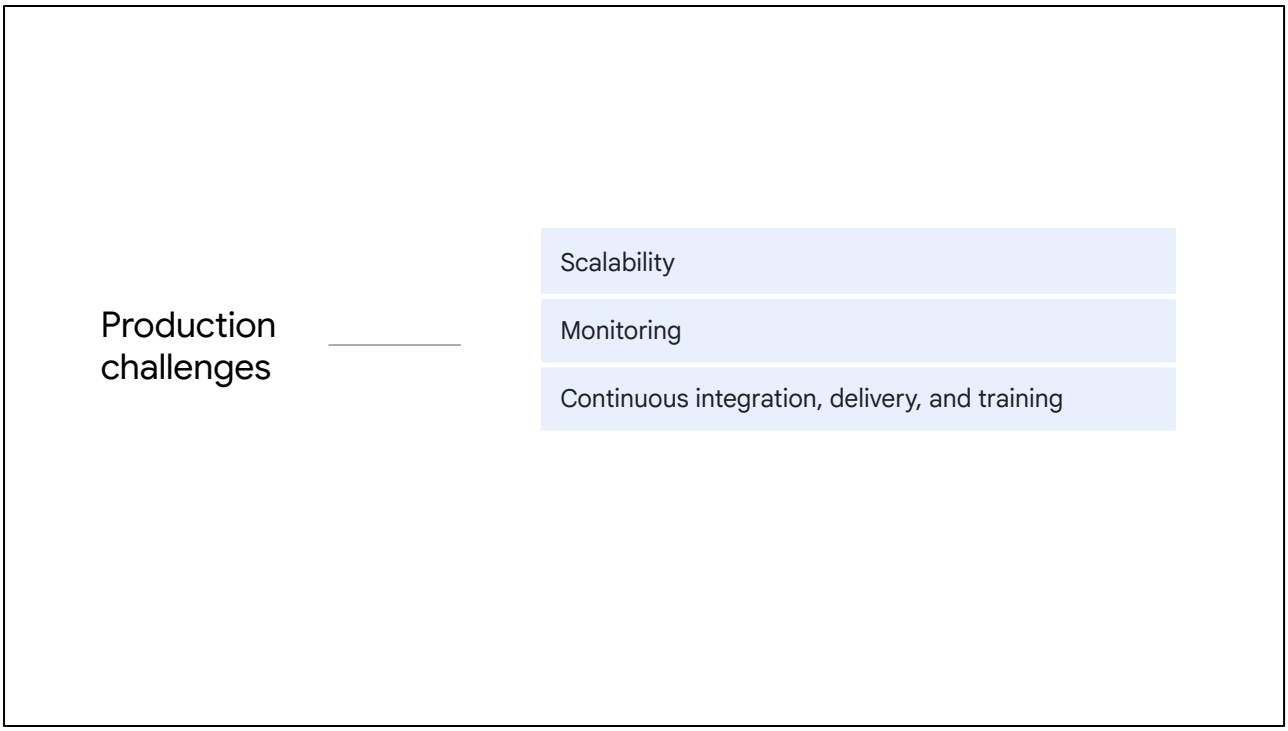
Before delving into these options, the next lesson will introduce Vertex AI, Google Cloud's AI development platform.



Before exploring various options of build an ML model, you first need to familiarize yourself with the playground, Vertex AI, Google's unified AI development platform.



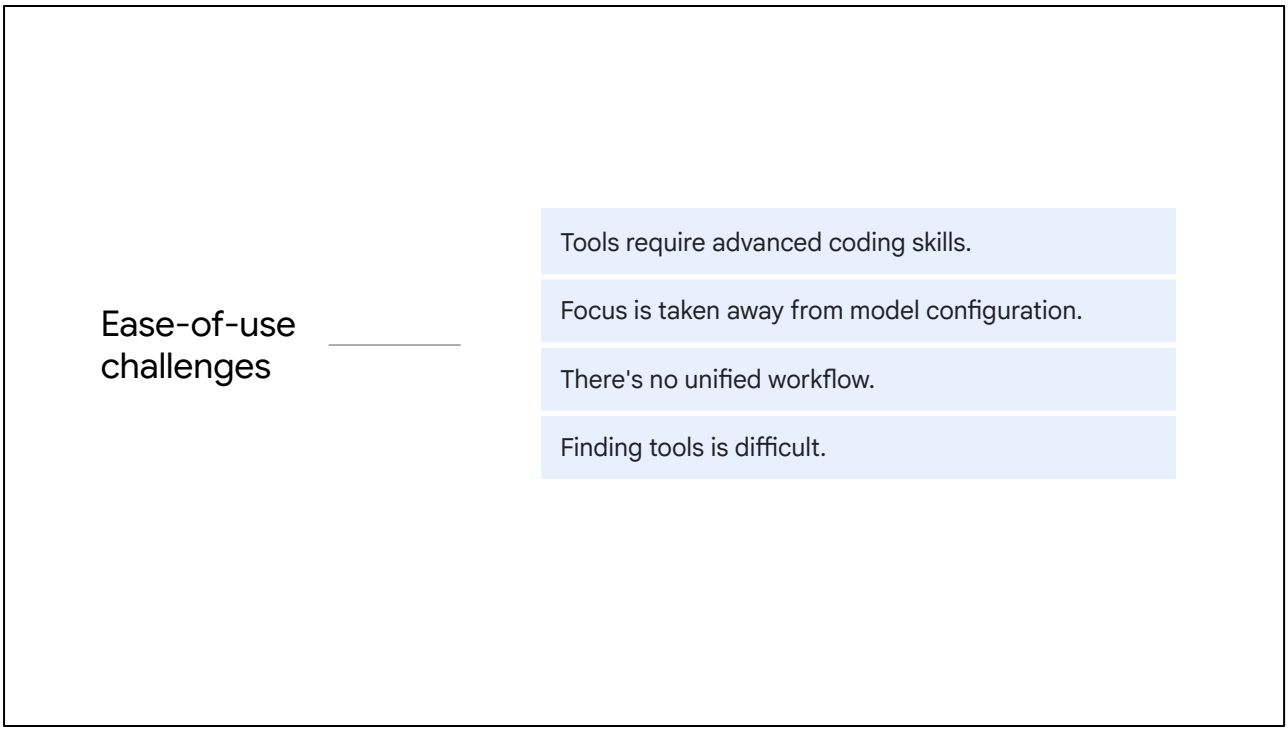
For years now, Google has invested time and resources into developing data and AI, and applied these technologies to many of its products and services, like Gmail, Google Maps, Google Photos, and Google Translate, just to name a few.



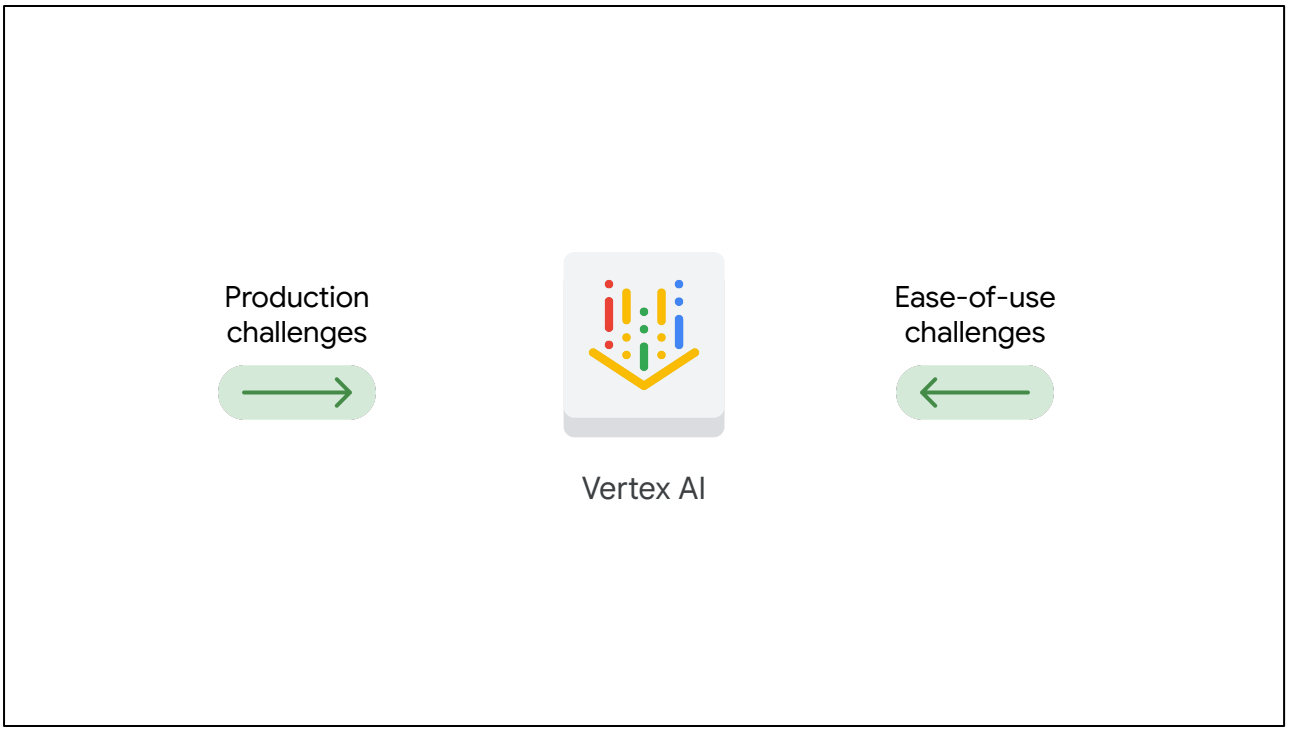
But developing these technologies doesn't come without challenges.

There are challenges around getting ML models into **production**. For example, scalability, monitoring, and continuous integration, delivery, and training.

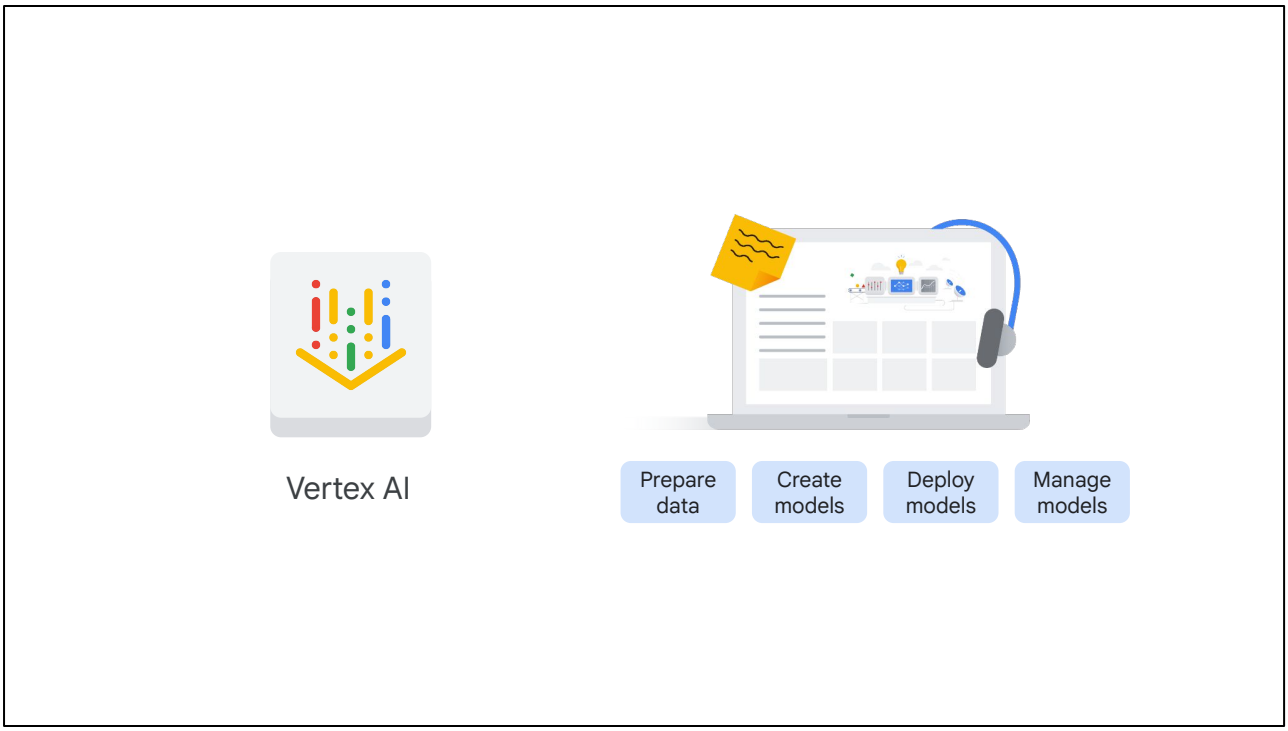
In fact, according to Gartner, only half of enterprise ML projects get past the pilot phase.



There are also **ease-of-use challenges**. Many tools on the market require advanced coding skills, which can take a data scientist's focus away from model configuration. Without a unified workflow, data scientists often have difficulties finding tools.

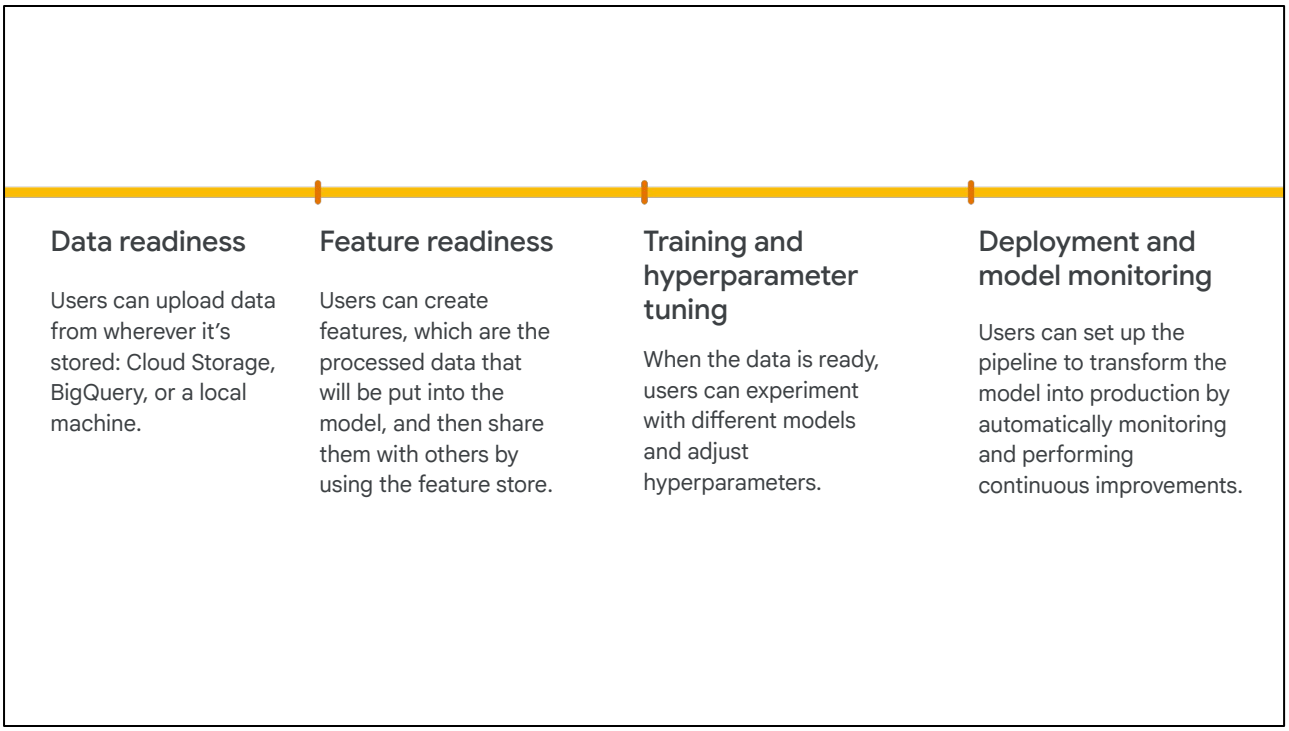


Google's solution to many of the **production** and **ease-of-use challenges** is **Vertex AI**, a unified platform that brings all the components of the machine learning ecosystem and workflow together.



So, what exactly does a unified platform mean? There are two primary aspects:

Firstly, it means that Vertex AI provides an end-to-end ML pipeline to prepare data, and create, deploy, and manage models over time and at scale.

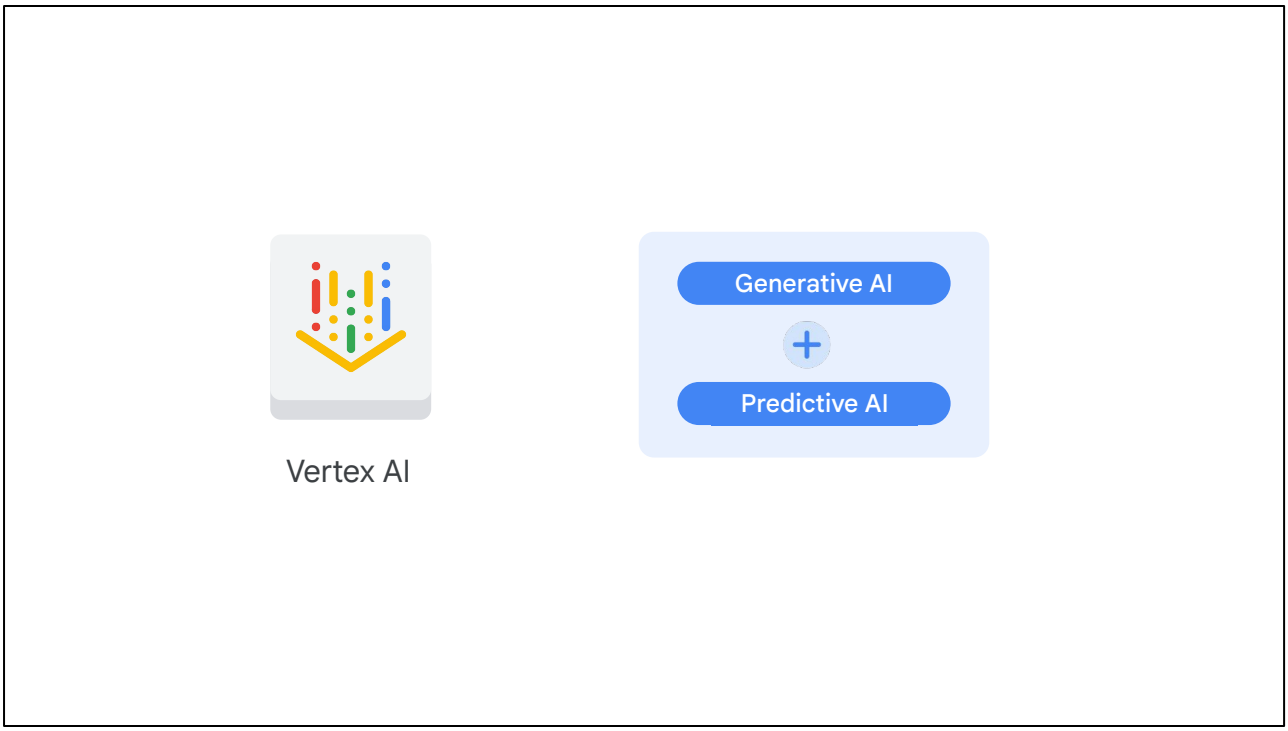


Firstly, it means that Vertex AI provides an end-to-end ML pipeline to prepare data, and create, deploy, and manage models over time, and at scale.

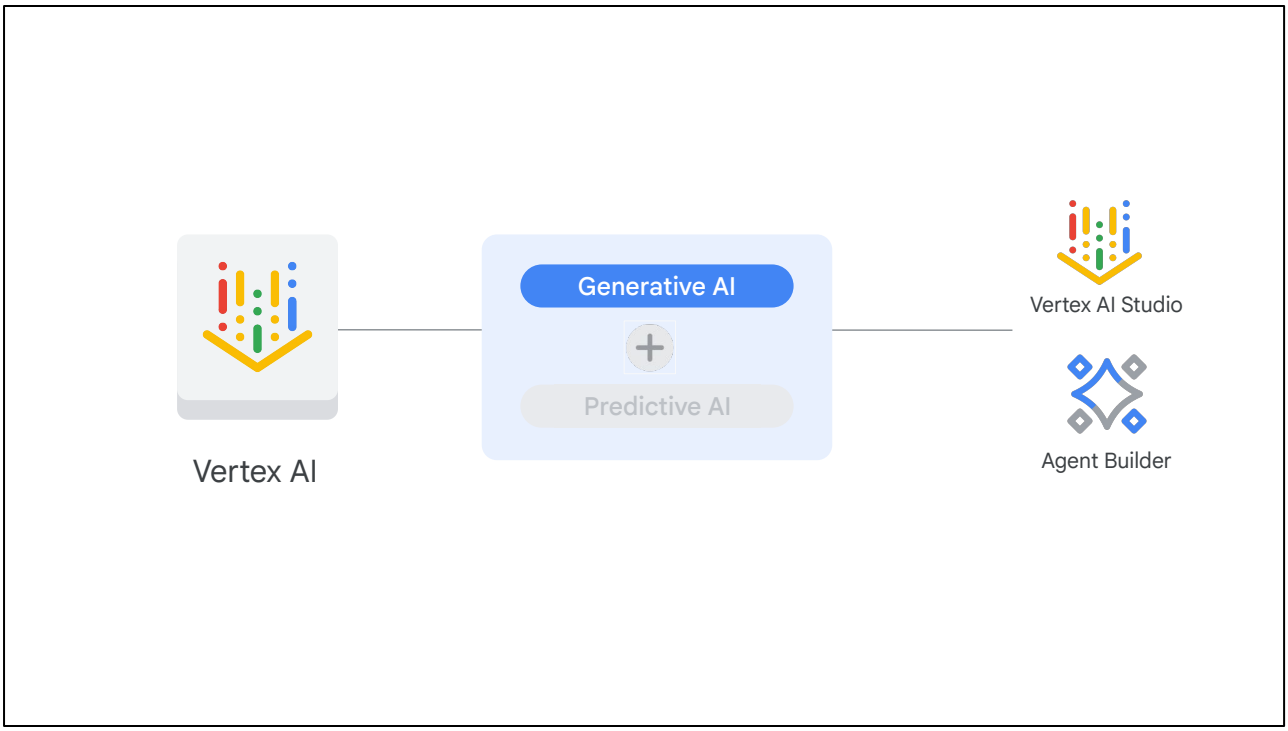
For instance,

- **During the data readiness** stage, users can upload data from wherever it's stored: Cloud Storage, BigQuery, or a local machine.
- Then, during the **feature readiness stage**, users can create features, which are the processed data that will be put into the model, and then share them with others by using the feature store.
- After that, it's time for **training and hyperparameter tuning**. This means that when the data is ready, users can experiment with different models and adjust hyperparameters.
- And finally, during **deployment and model monitoring**, users can set up the **pipeline** to transform the model into production by automatically monitoring and performing continuous improvements.

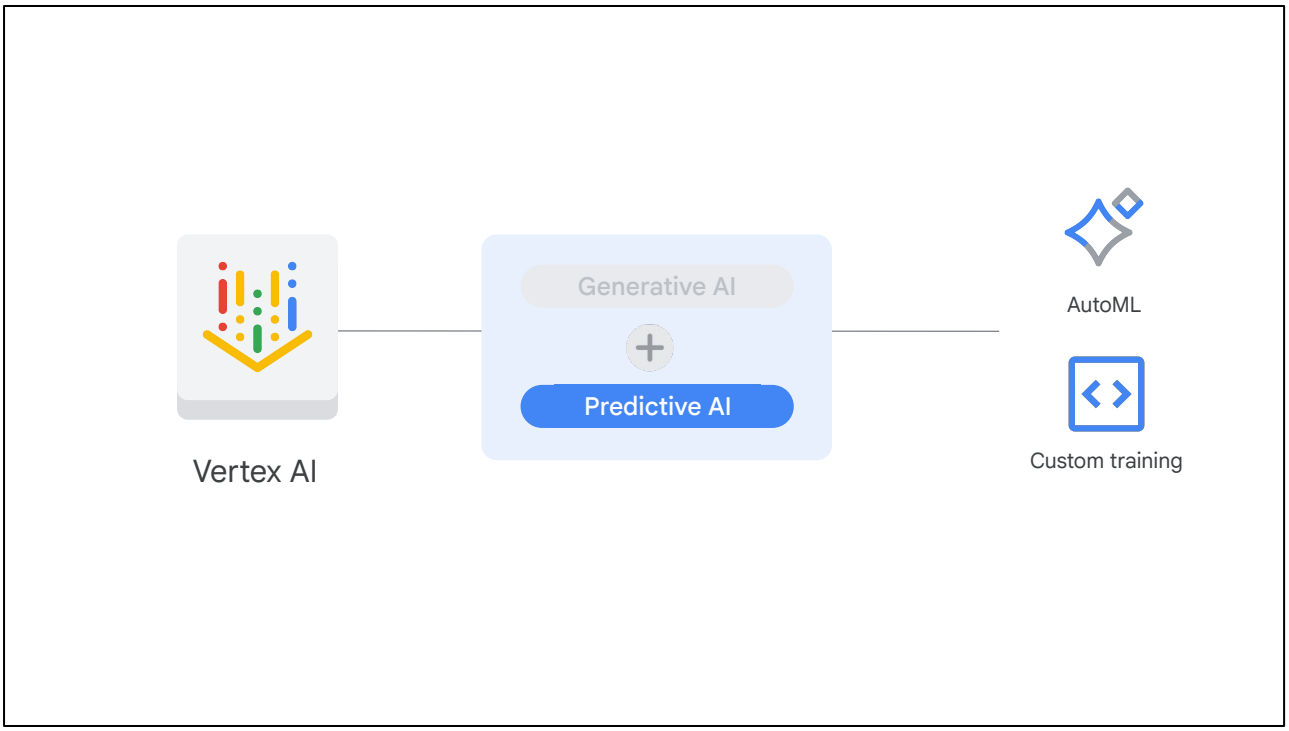
You'll learn how to do this later in the course, when you explore MLOps.



Second, Vertex AI is a unified platform that encompasses both **generative AI,** enabling creation of multimodal content, and **predictive AI**, allowing for forecasting and classification.



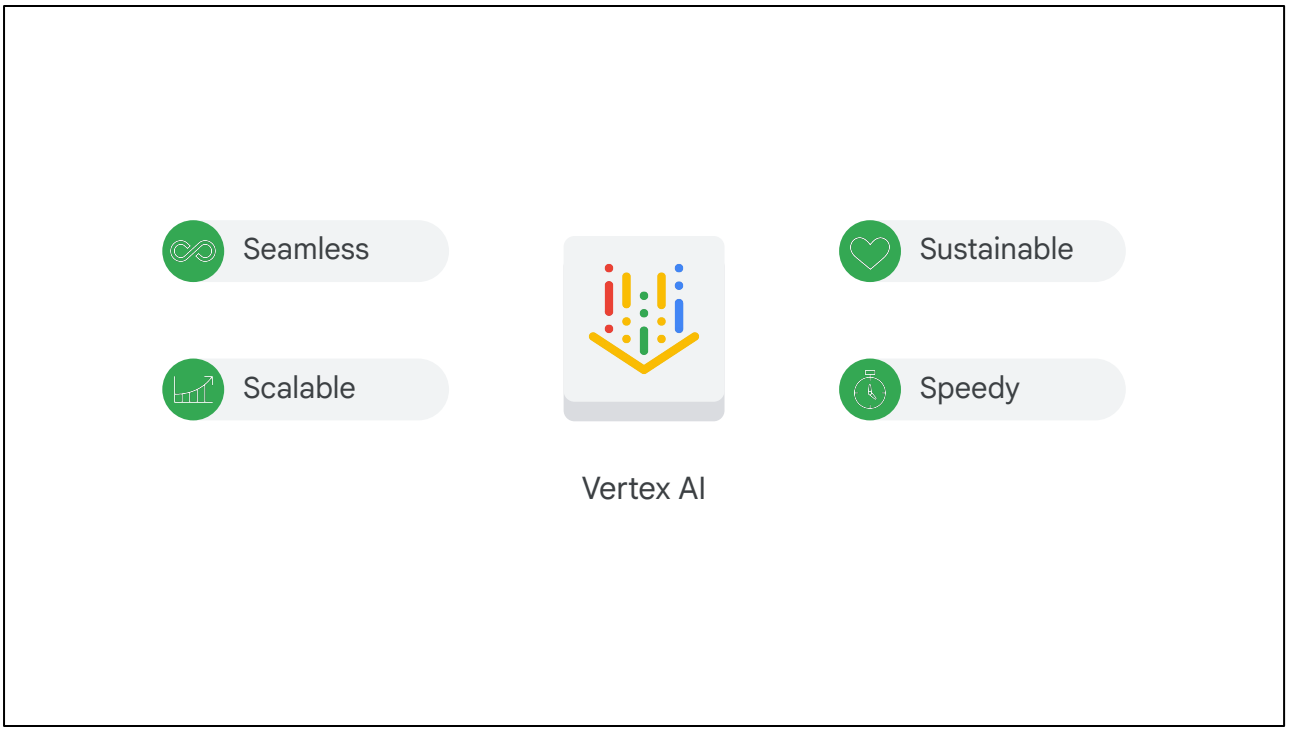
You already explored gen AI tools in the previous module, like Vertex AI Studio and Agent Builder.



Let's focus on the tools for predictive AI. Vertex AI allows users to build ML models with either **AutoML**, a no-code solution, or **custom training**, a code-based solution.

- AutoML provides a UI that is easy to navigate. It lets data scientists focus on what business problems to solve instead of how to code and deploy an ML solution.
- Custom training gives data scientists and ML engineers more control over the development environment and process. They can use tools like Vertex AI Workbench and Colab to do their ML projects themselves.

One convenient feature is that data scientists can now write SQL with Workbench on Vertex AI to seamlessly connect BigQuery and Vertex AI.

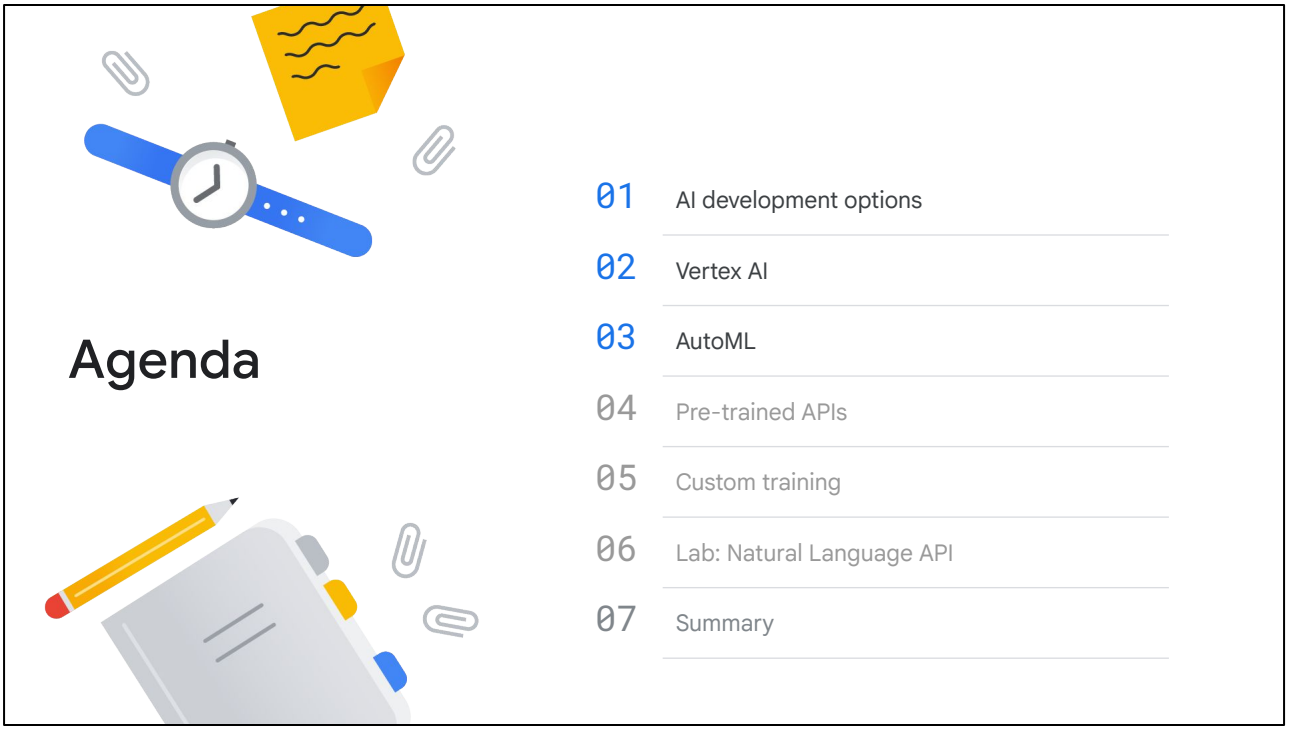


Being able to perform such a wide range of tasks in one unified platform has many benefits.

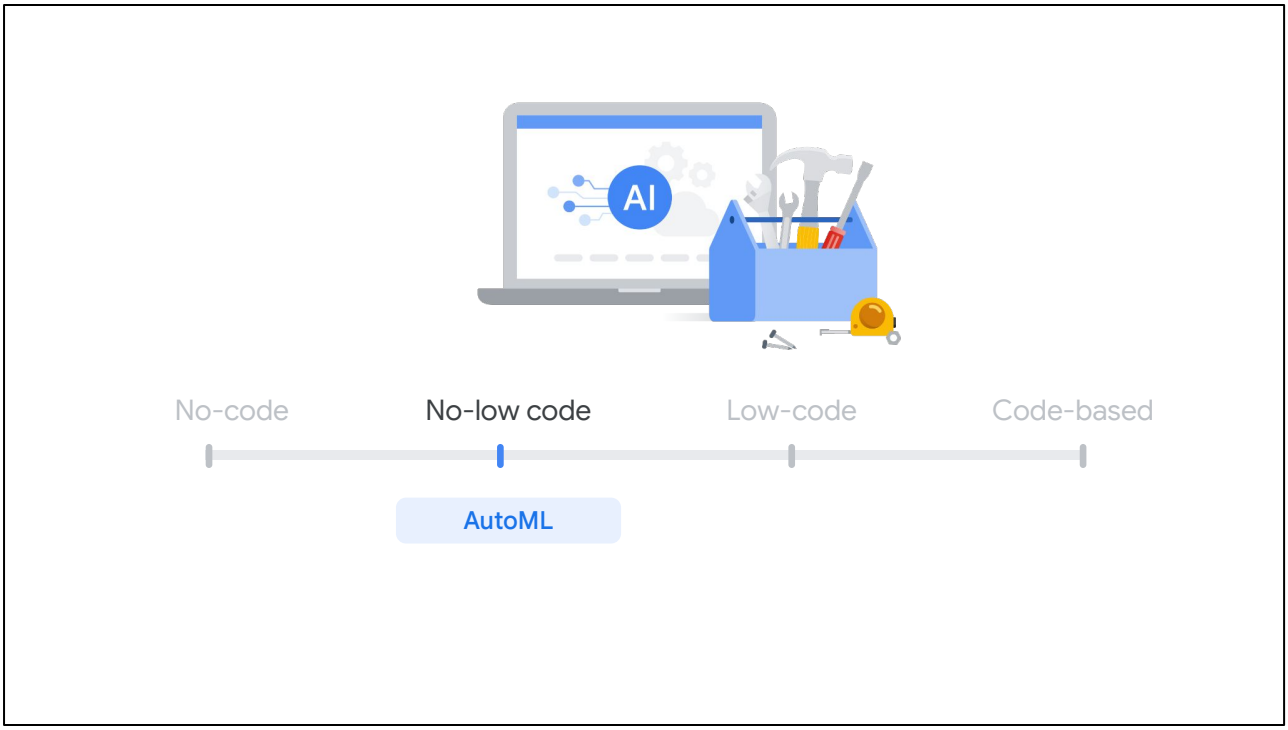
This can be summarized with four Ss:

- It's **seamless**. Vertex AI provides a smooth user experience from uploading and preparing data all the way to model training and production.
- It's **scalable**. The machine learning operations (MLOps) provided by Vertex AI help to monitor and manage the ML production and therefore scale the storage and computing power automatically.
- It's **sustainable**. All of the artifacts and features created using Vertex AI can be reused and shared.
- And it's **speedy**. Vertex AI produces models that have 80% fewer lines of code than competitors.

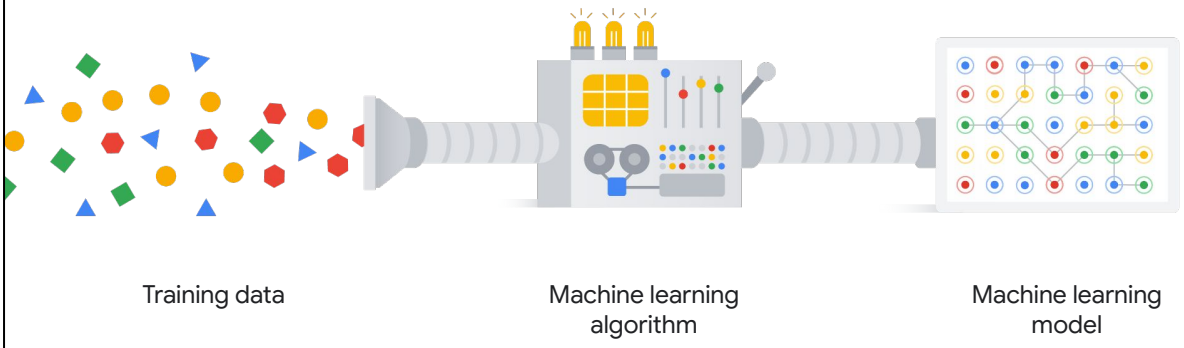
Let's take a deep dive into ML model building options in the next lesson!



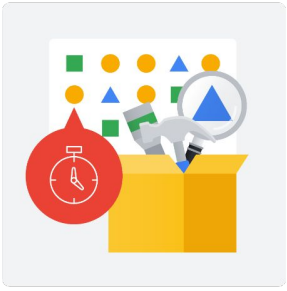
In the previous lesson, you learned about Vertex AI, a unified platform that supports both AutoML, a no-code solution, and custom training, a code-based solution.



In this lesson, you'll explore AutoML in depth, including the technologies used to power automated ML development.



AutoML, which stands for automated machine learning, aims to automate the process to develop and deploy an ML model.

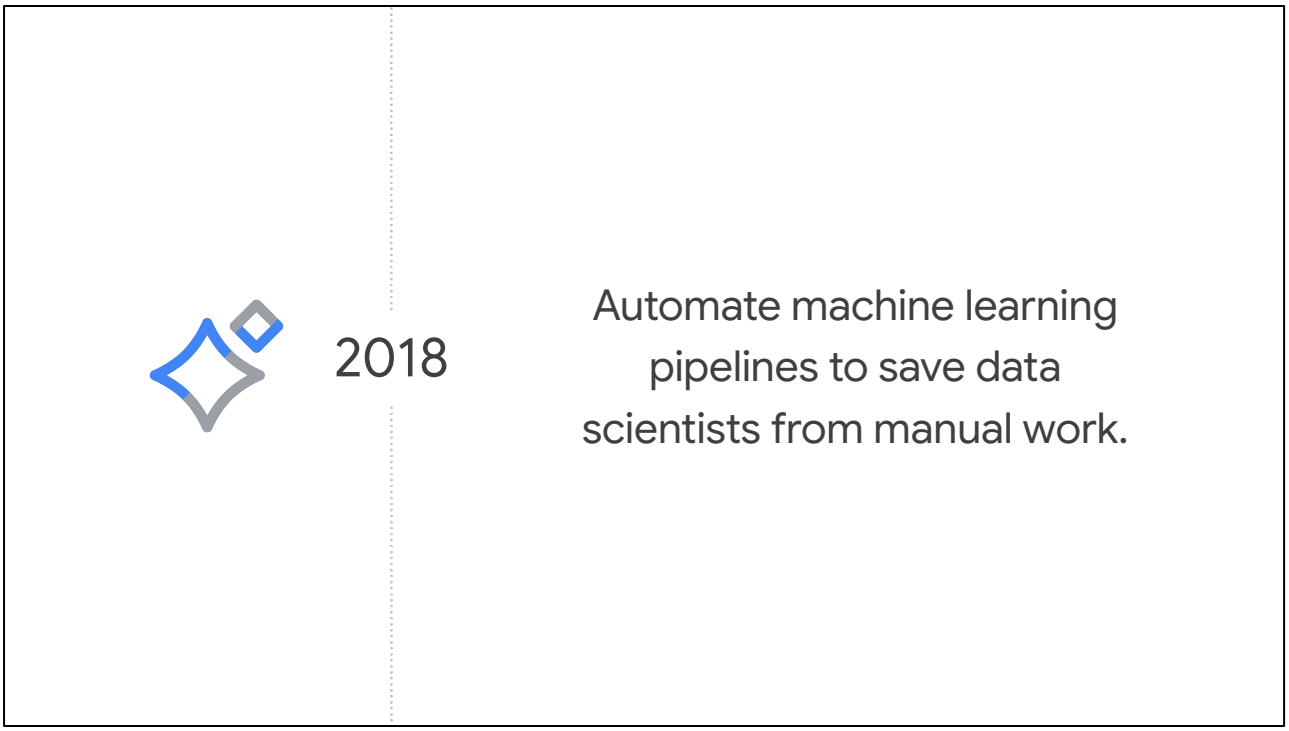


Repeatedly add new data and features.

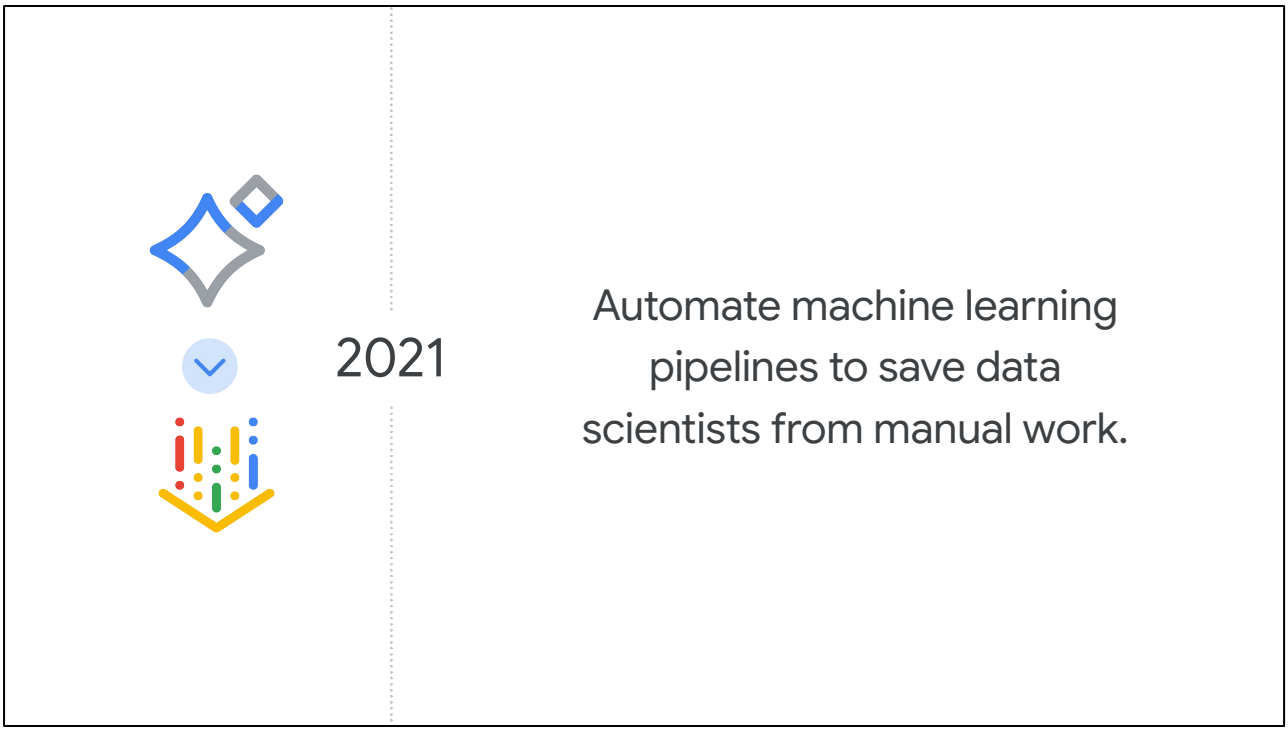
Try different models.

Tune parameters.

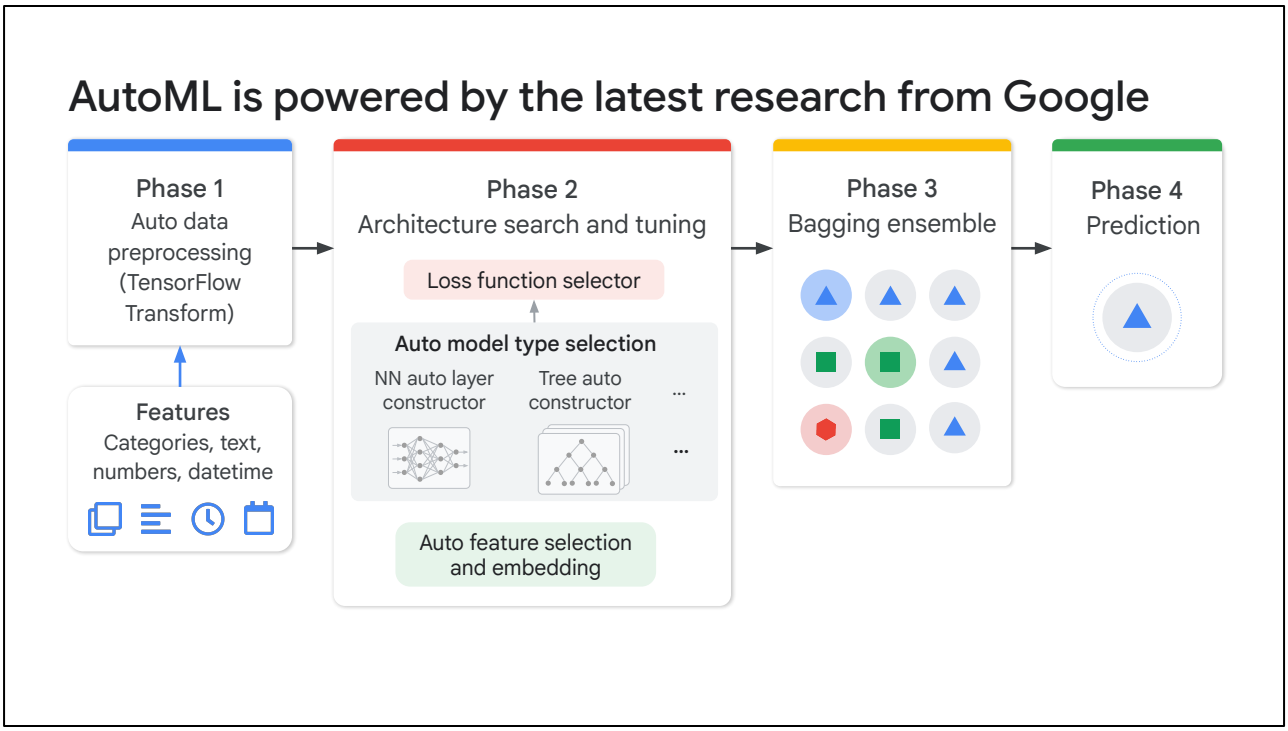
If you've worked with ML models before, you know that building them can be extremely time consuming, because you need to repeatedly add new data and features, try different models, and tune parameters to achieve the best result.



When AutoML was first announced in January of 2018, the goal was to save the manual work from data scientists and automate machine learning pipelines from pre-processing data to model training and deployment.



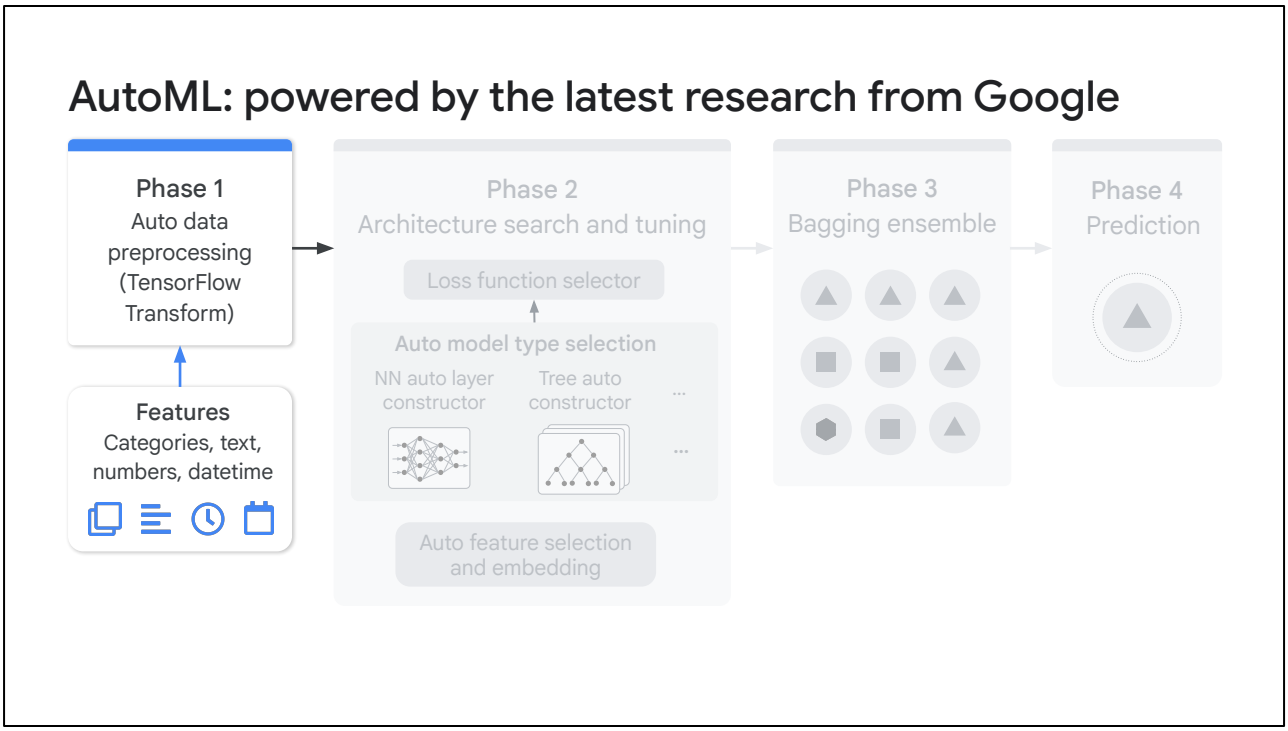
Since 2021, AutoML features are embedded in Vertex AI and have become part of the platform.



But how could this be done? How can you trust AutoML to generate the best results without bias and fast?

Let's look deeper to explore how AutoML works and the main technologies behind it.

AutoML is powered by the latest research from Google. It's an ongoing endeavor.

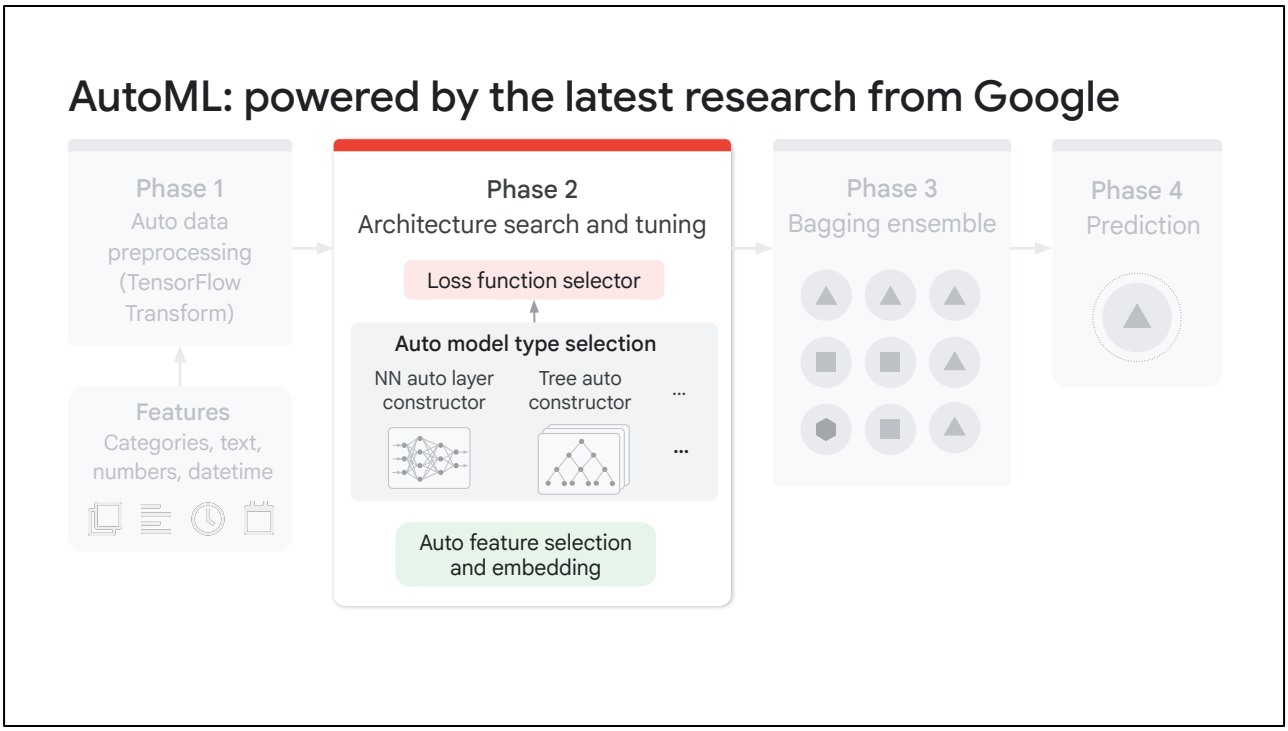


There are four distinct phases. Phase one is data processing. After you upload a dataset, AutoML provides functions to automate part of the data preparation process.

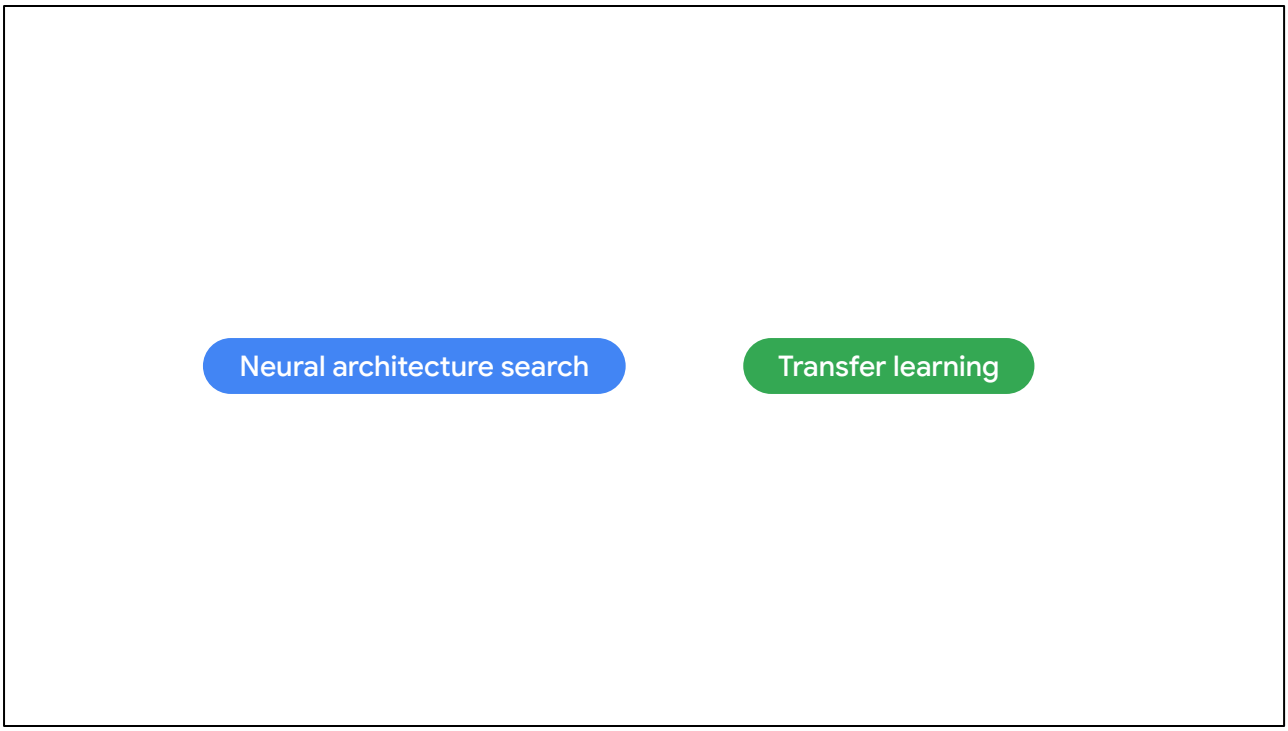
Automated feature engineering

- **Numbers:** Generate quantiles, log, z_score transforms.
- **Categories:** Use one-hot encoding, grouping, embeddings.
- **Datetime:** Extract year, month, day, weekday; categorize.
- **Arrays of categories:** Convert to lookup index, generate embeddings.
- **Text:** Tokenize, generate n-grams, create embeddings.
- **Nested fields:** Flatten, apply type transformations.

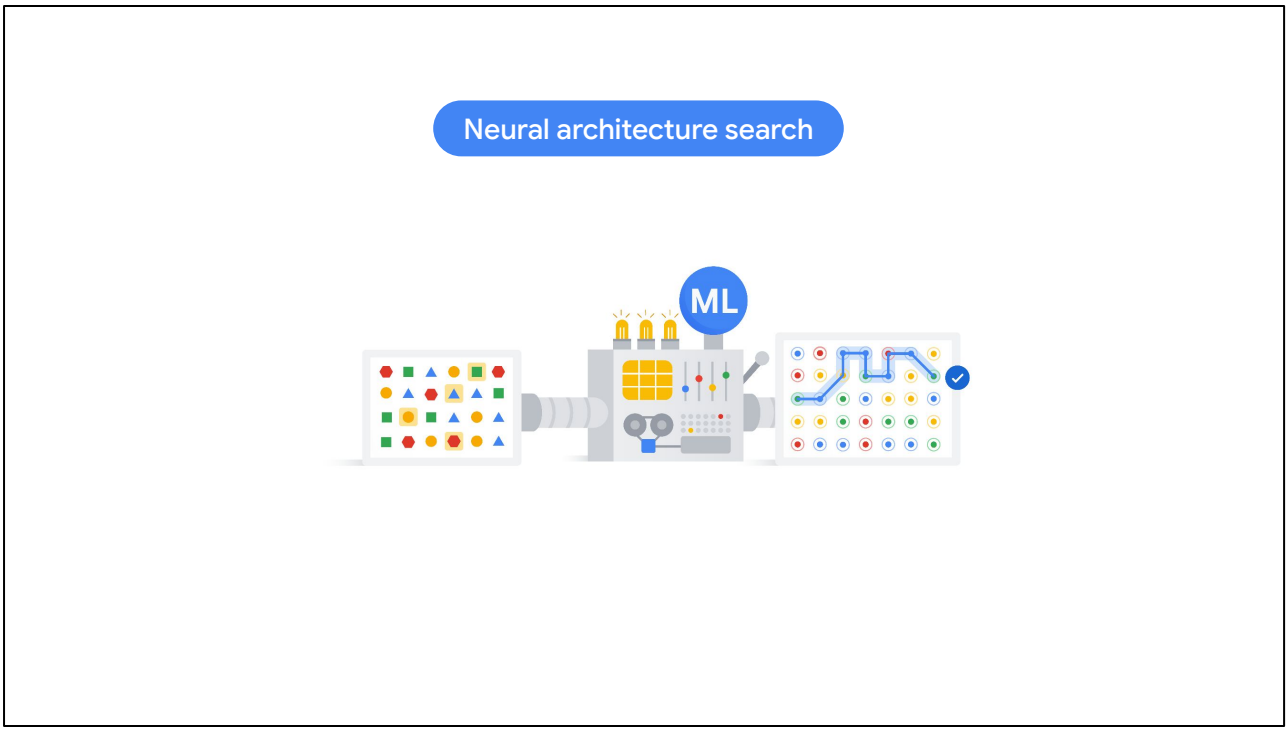
For example, it can covert numbers, datetime, text, categories, arrays of categories, and nested fields into a certain format of data so that it can be fed into an ML model.



Phase two includes searching the best models and tuning the parameters.



Two critical technologies support this auto search. The first one is called **neural architecture search**, which helps search the best models and tune the parameters automatically. And the second one is called **transfer learning**, which helps speed the search by using the pre-trained models.

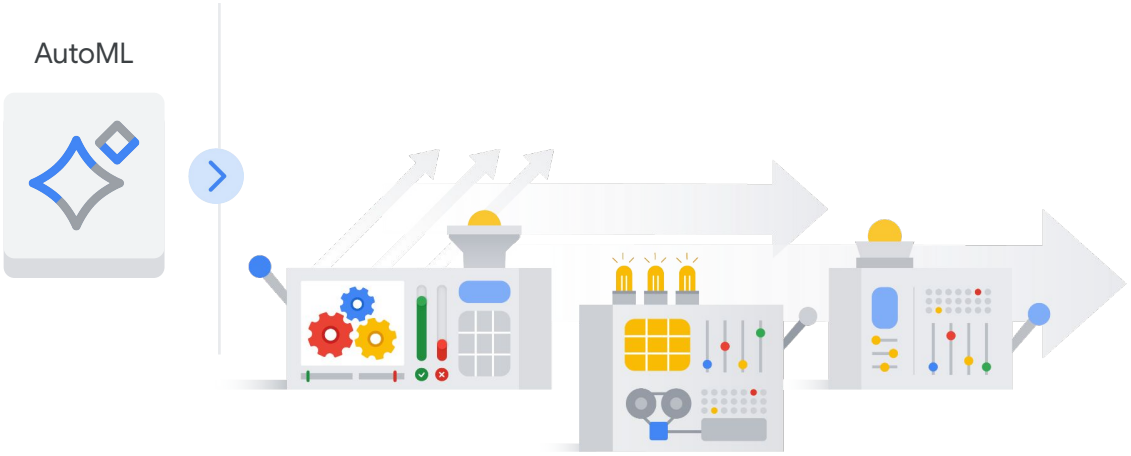


Let's first look at **neural architecture search**.

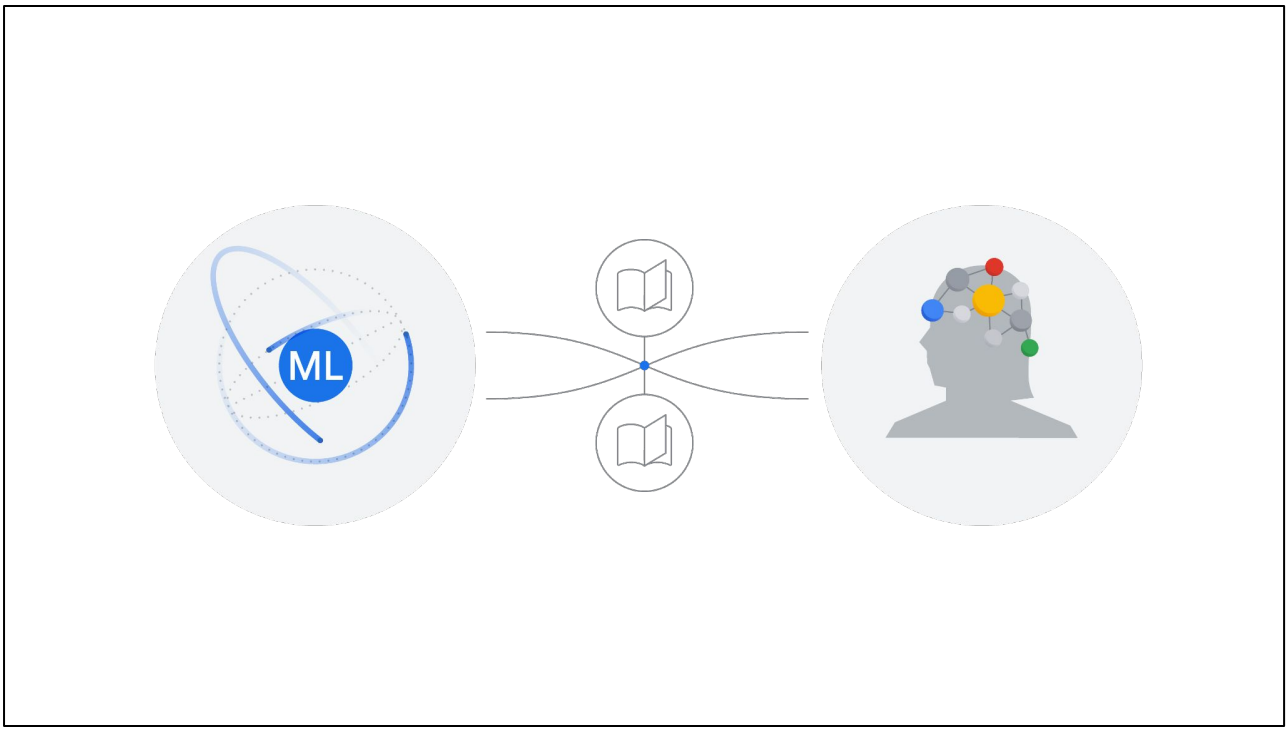
The goal of neural architecture search is to find optimal models among many options. Specifically, AutoML tries different architectures and models, and compares the performance of the models to find the best ones.

Automated model architecture search

Search through multiple ML models, and automatically tune the parameters.

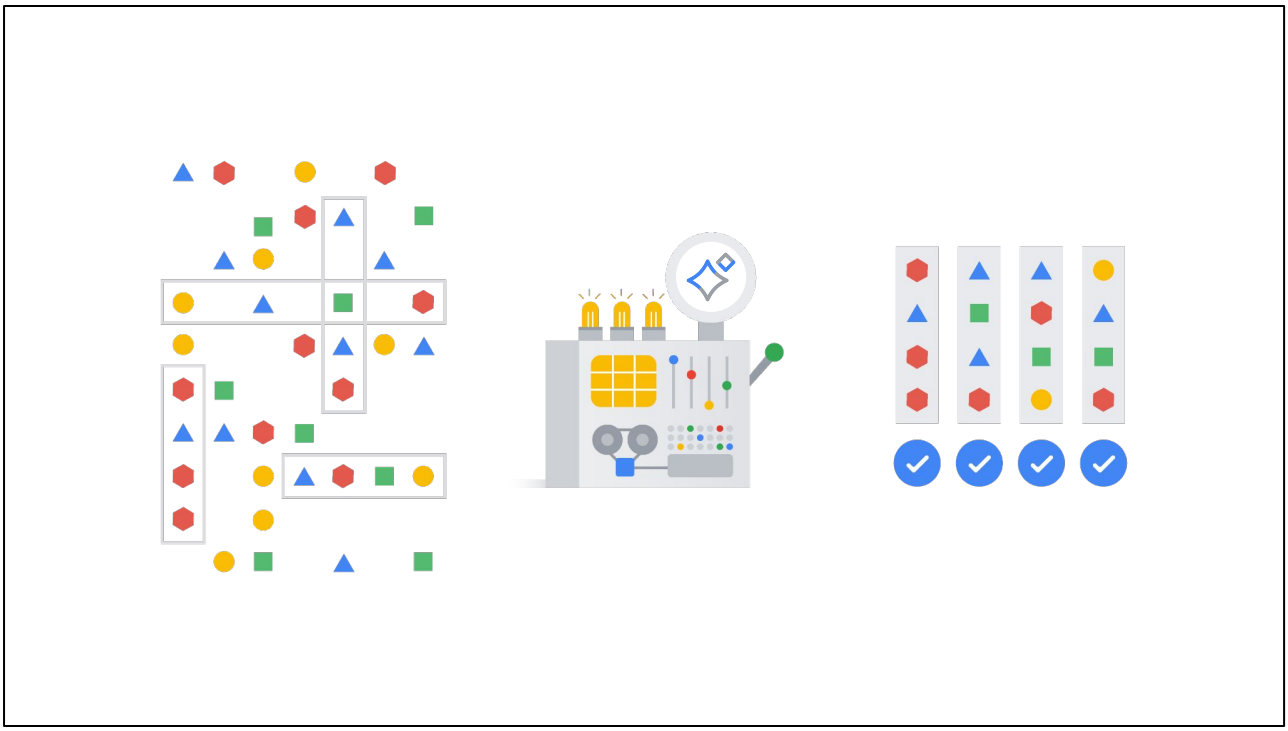


For instance, AutoML can search through multiple advanced ML models and automatically tune the parameters to find the best fit for your data.

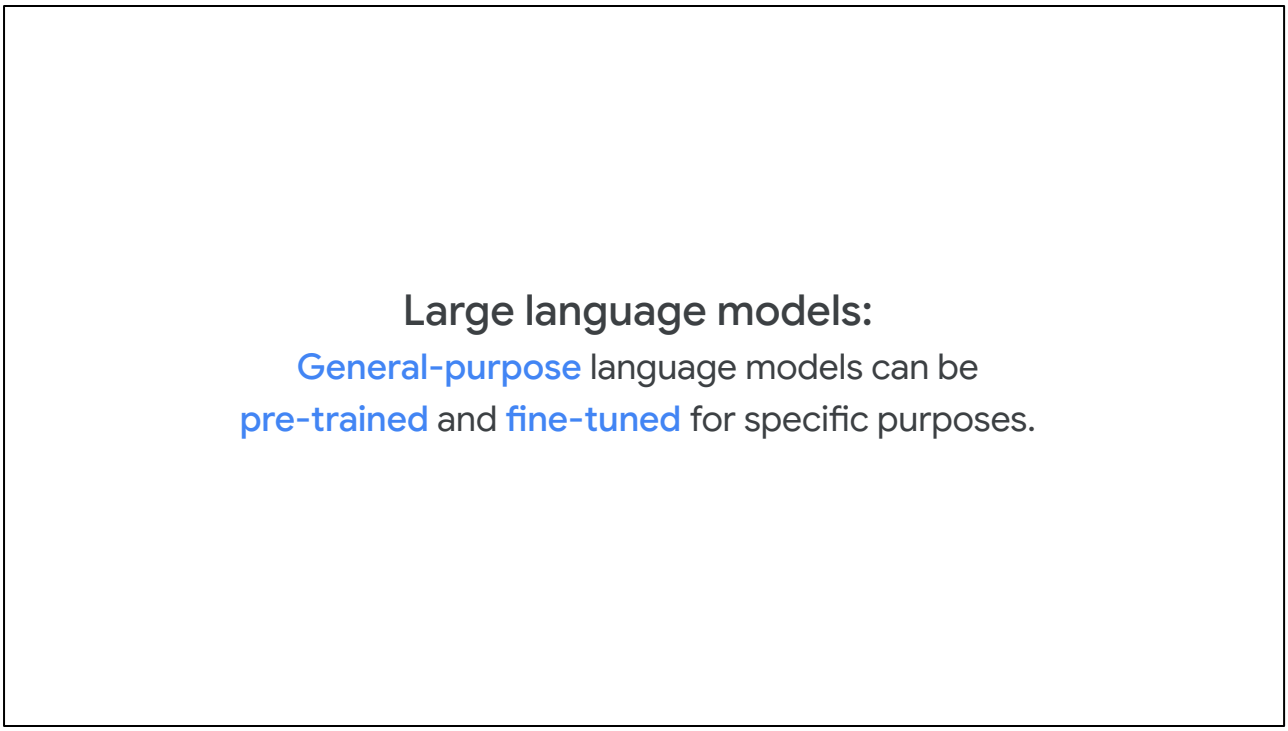


Secondly, let's examine **transfer learning**.

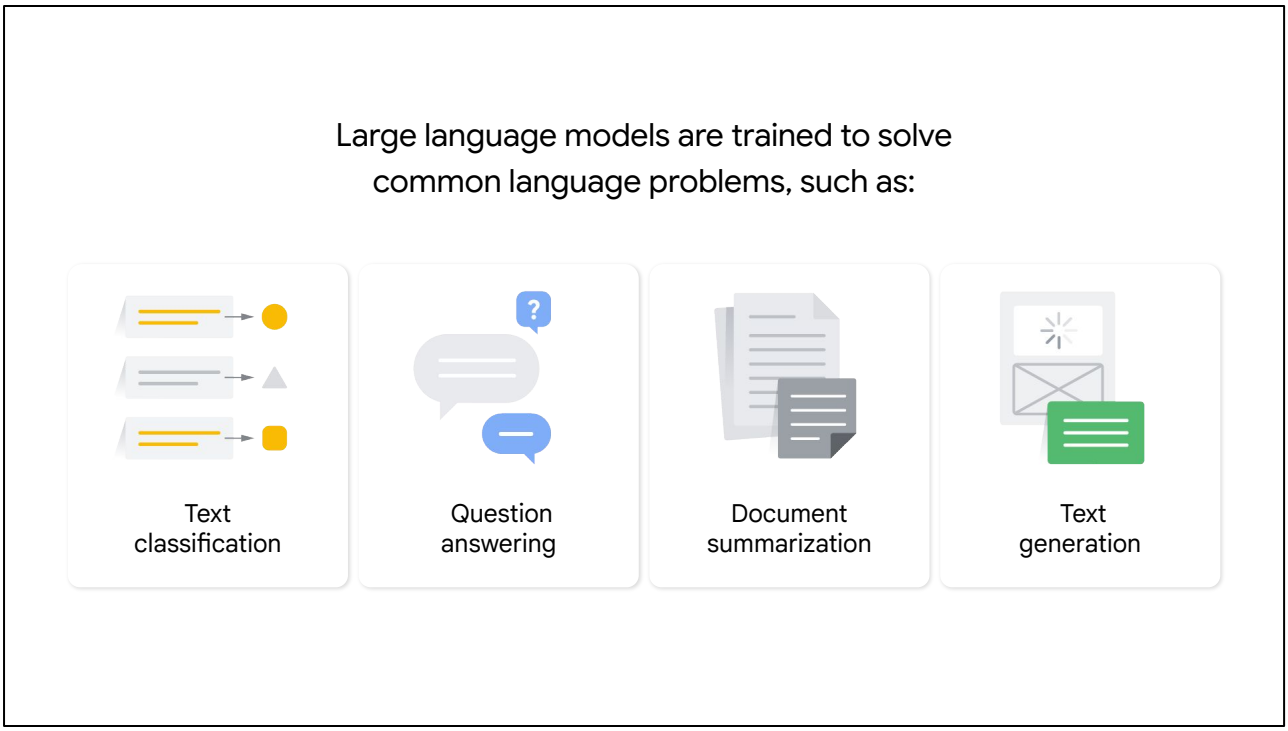
Machine learning is similar to human learning. It learns new things based on existing knowledge.



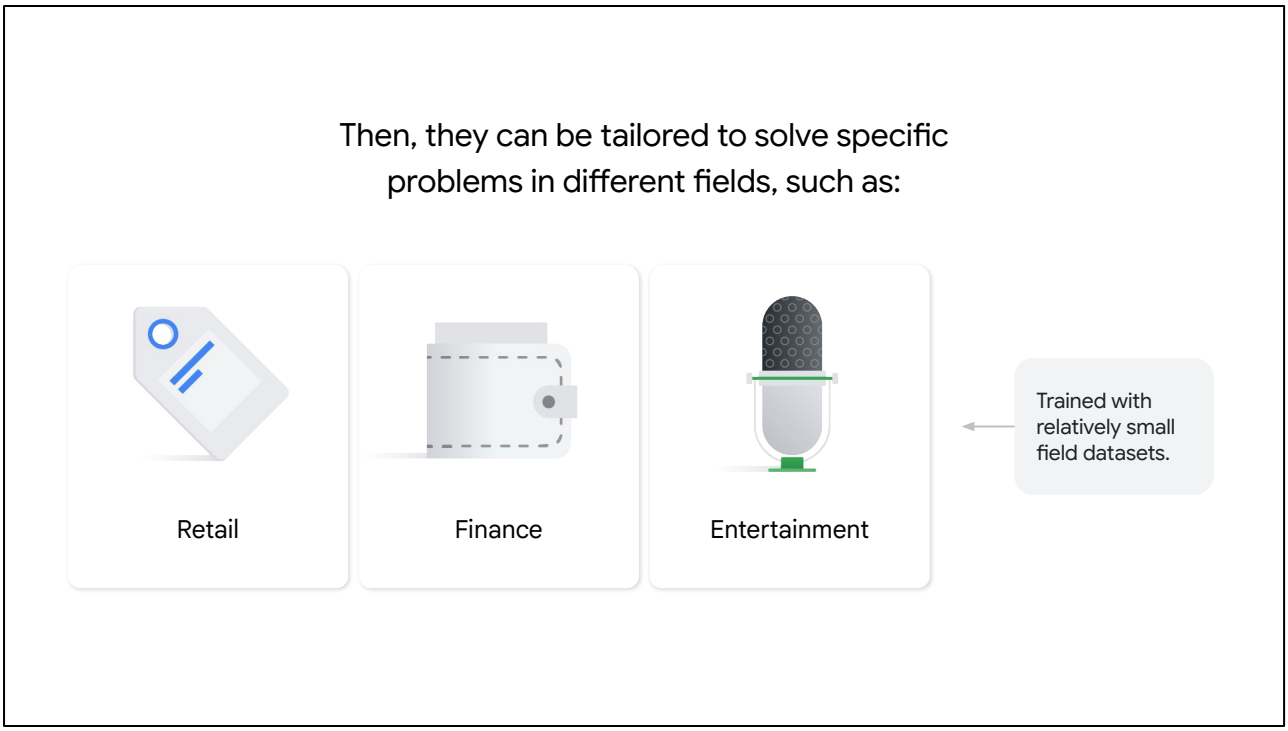
AutoML has already trained many different models with large amounts of data. These trained models can be used as a foundation model to solve new problems with new data.



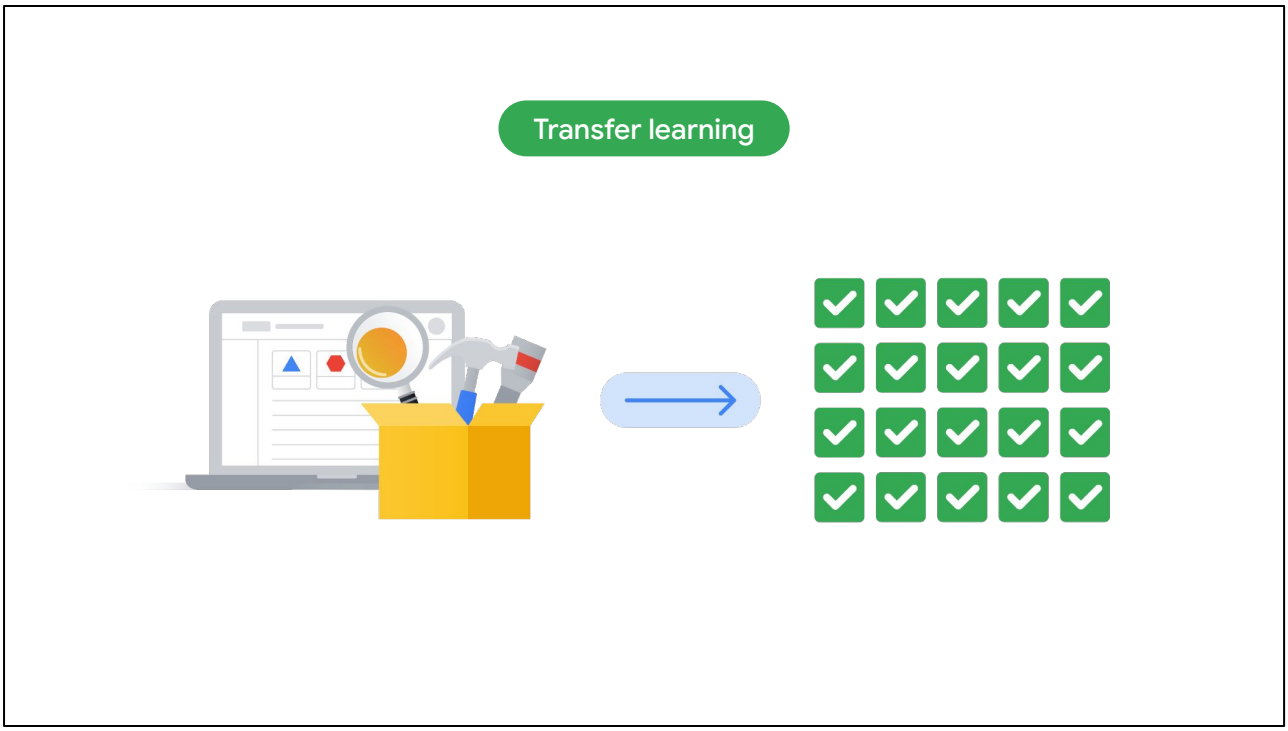
A typical example are large language models (LLM), which are general purpose and can be **pre-trained** and **fine-tuned** for specific purposes.



LLMs are trained for general purposes to solve common language problems such as text classification, question answering, document summarization, and text generation across industries.



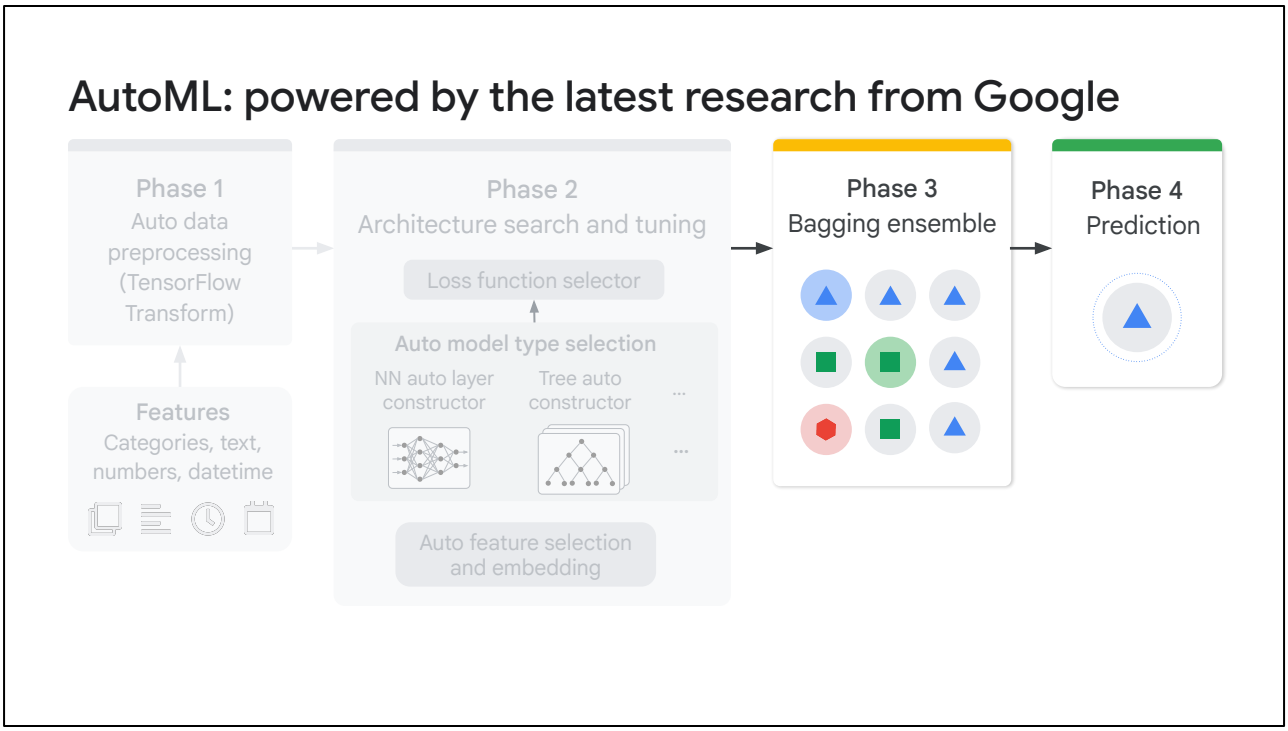
The models can then be tailored to solve specific problems in different fields such as retail, finance, and entertainment, using a relatively small size of field datasets.



Transfer learning is a powerful technique that lets people with smaller datasets or less computational power achieve great results by using pre-trained models trained on similar, larger datasets.

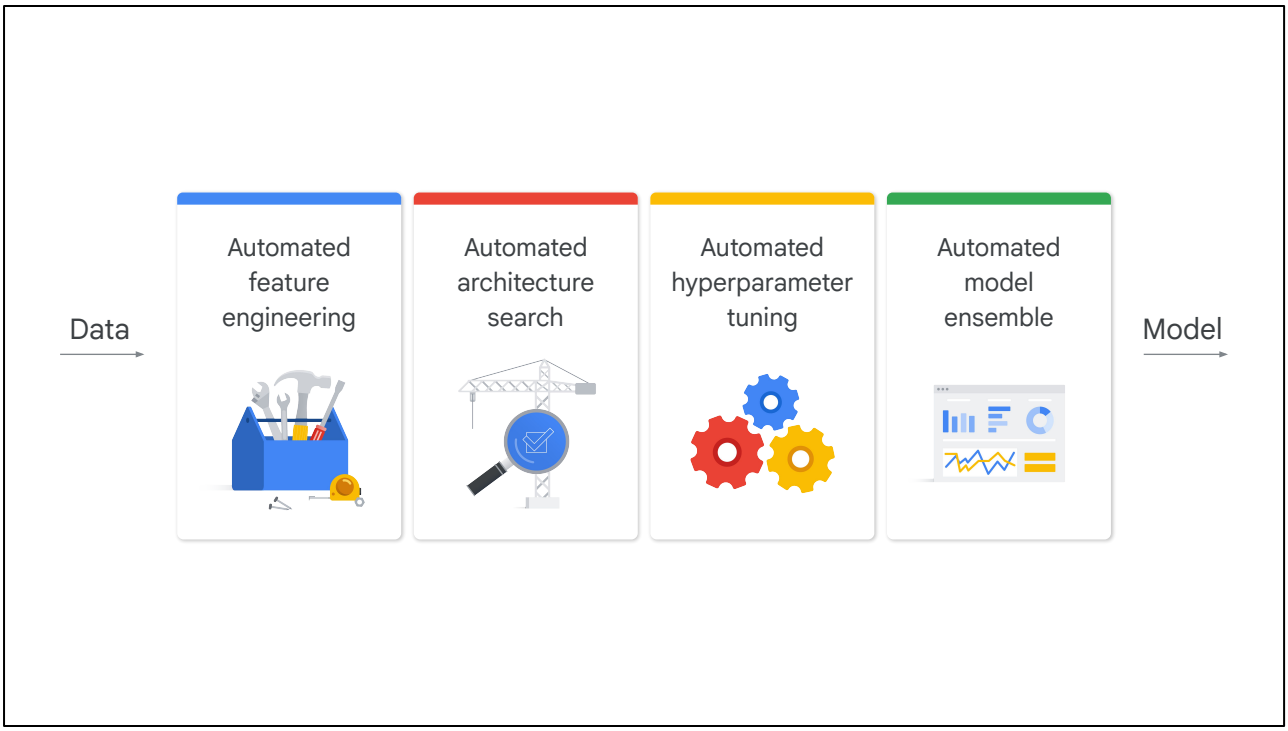
Because the model learns through transfer learning, it doesn't have to learn from the beginning.

So it can generally reach higher accuracy with much less data and computation time than models that don't use transfer learning.



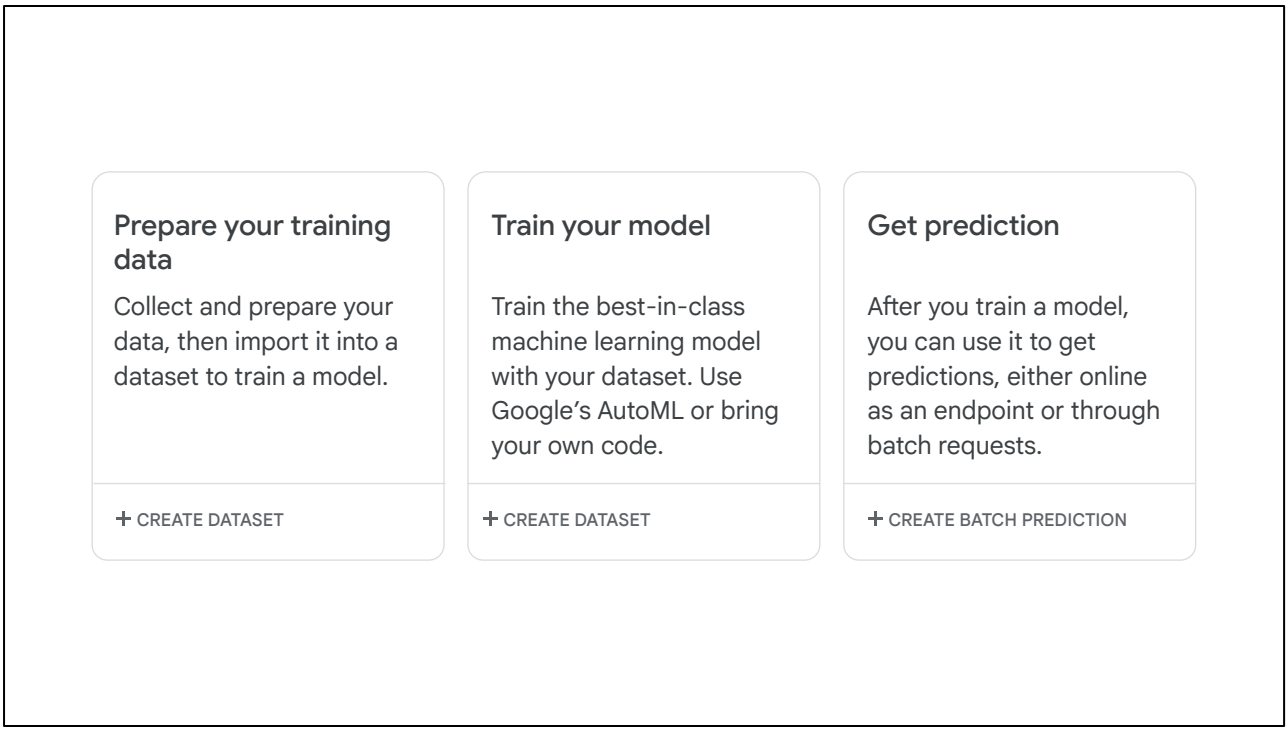
In phase three, the best models are assembled from phase 2 and prepared for prediction in phase 4.

Note that AutoML does not rely on one single model, but on the top number of models. The number of models depends on the training budget, but is typically around ten. The assembly can be as simple as averaging the predictions of the top number of models. Relying on multiple top models instead of one greatly improves the accuracy of prediction.

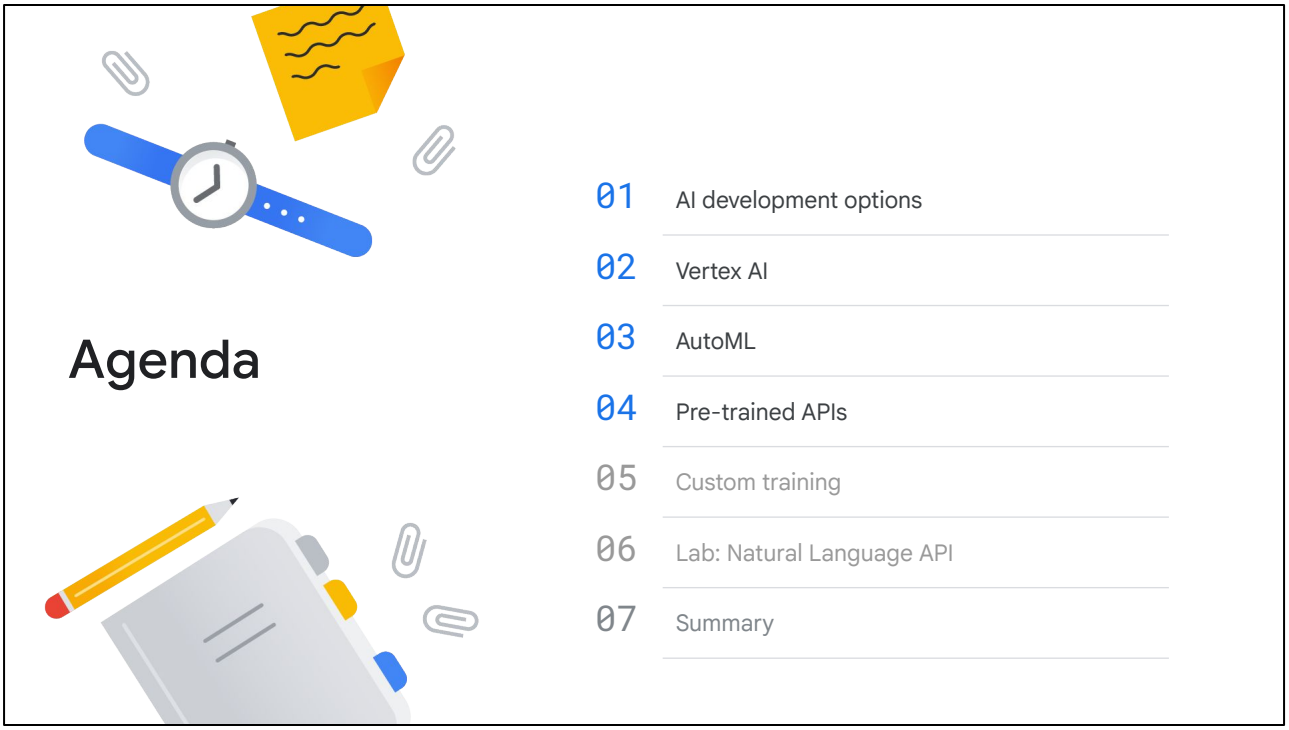


By applying these advanced ML technologies, AutoML automates the pipeline from feature engineering, to architecture search, to hyperparameter tuning, and to model ensemble.

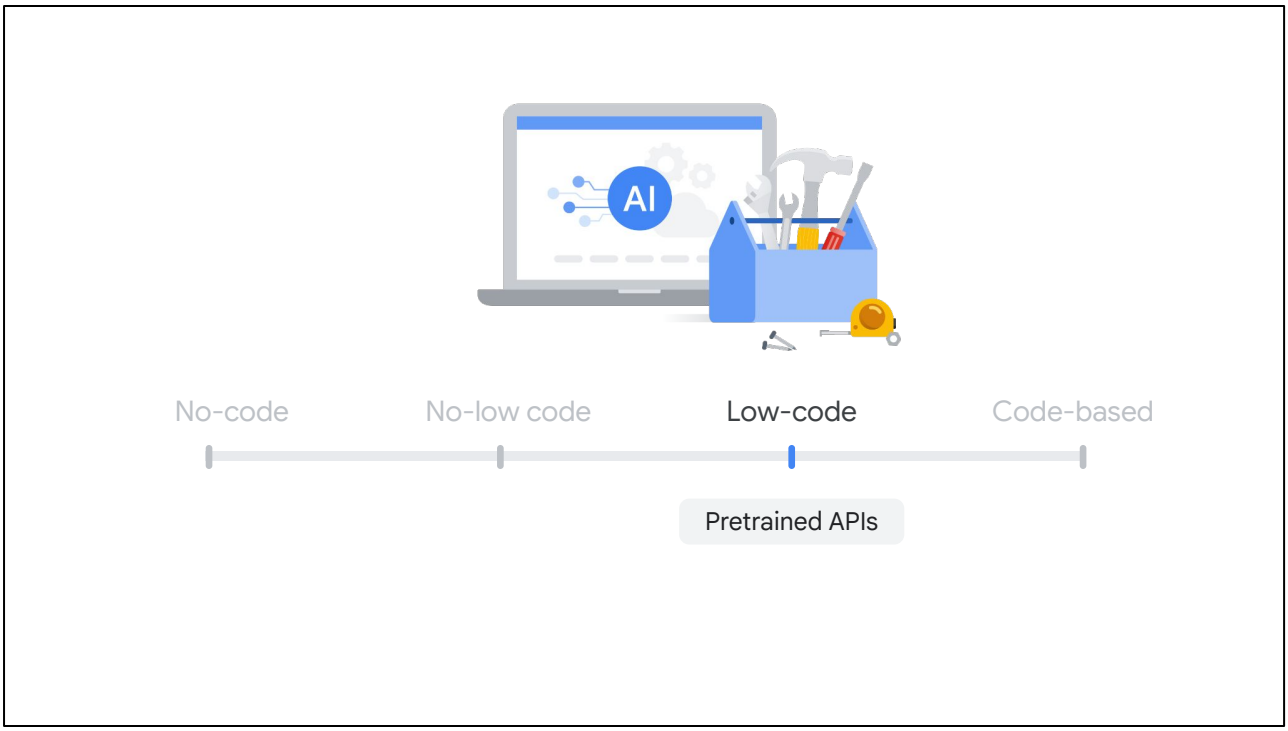
It might seem that AutoML can do a better job than a human to find the optimal models that fit your data.



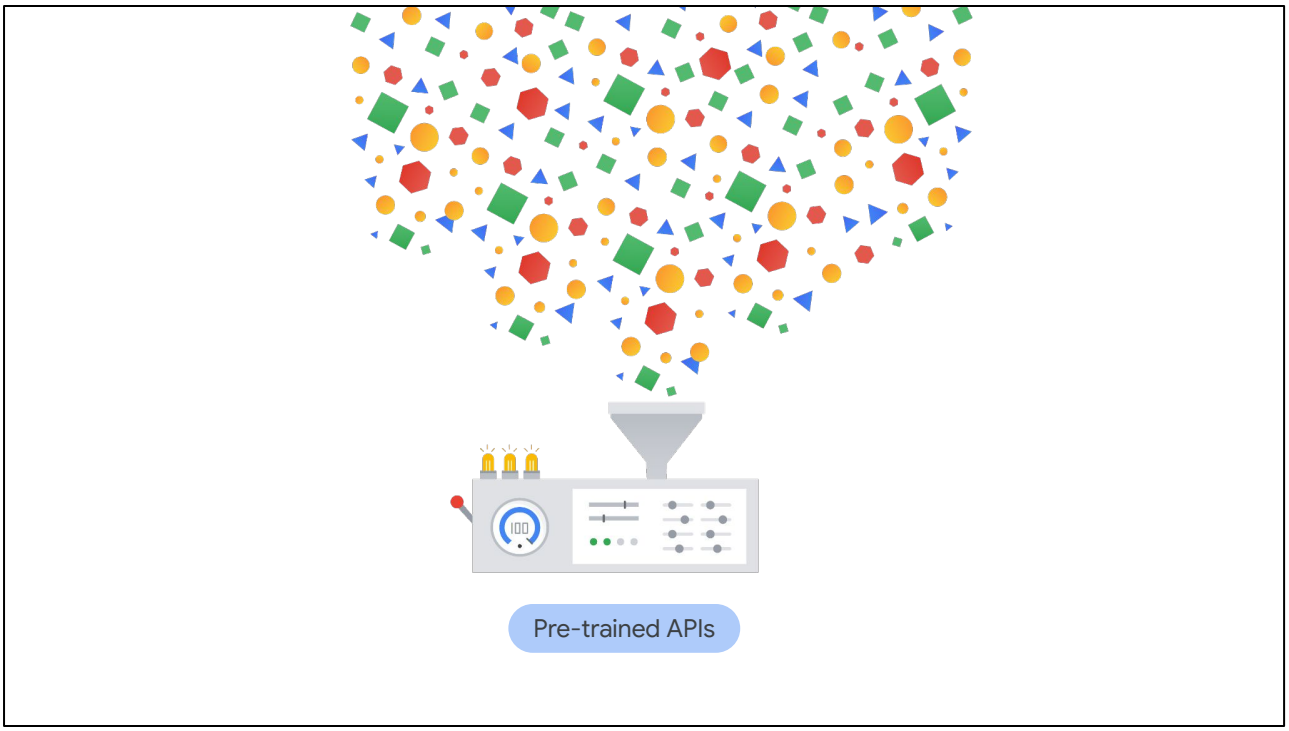
Perhaps, the best feature of AutoML is that it provides a **no-code solution**. You can point and click through a UI to build an ML model with your own data. You'll walk through the details from preparing training data, to train you model, and finally get prediction in the next module.



In the previous lesson, you learned about no- to low-code solution of AutoML.



Let’s proceed to a low-code solution by using pre-trained APIs.



Good machine learning models require lots of high-quality training data.

You should aim for hundreds of thousands of records to train a custom model. But what if you don't have that kind of data? How can you use AI to serve your purposes?

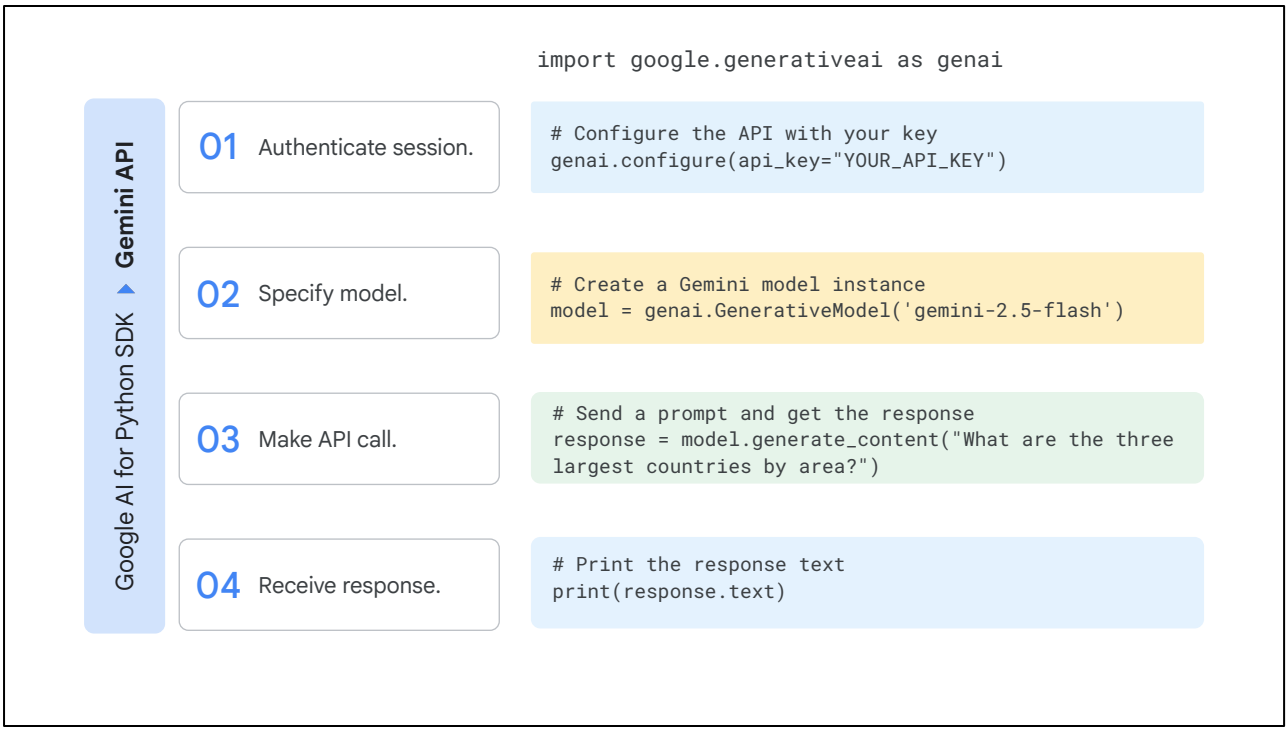
Pre-trained APIs are a great place to start.

Type A Type B Type F

API stands for application programming interface. APIs define how software components communicate with each other.

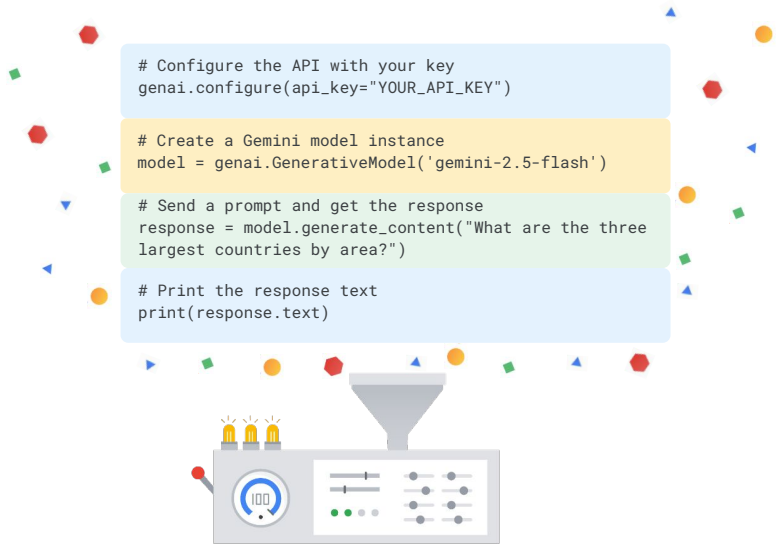
Imagine APIs as electrical outlets. Different regions have different standards. For example, the US uses type A and B, whereas Europe uses type F. As a traveler, you only need to know which adapter to use without worrying about what's behind the wall or how the electrical network is built.

The same principle applies to APIs. As a user, you only need to know which API to use and what parameters to pass, and in what format, without worrying about the implementation—specifically, the intricacies of model training and deployment—much like calling a predefined function.

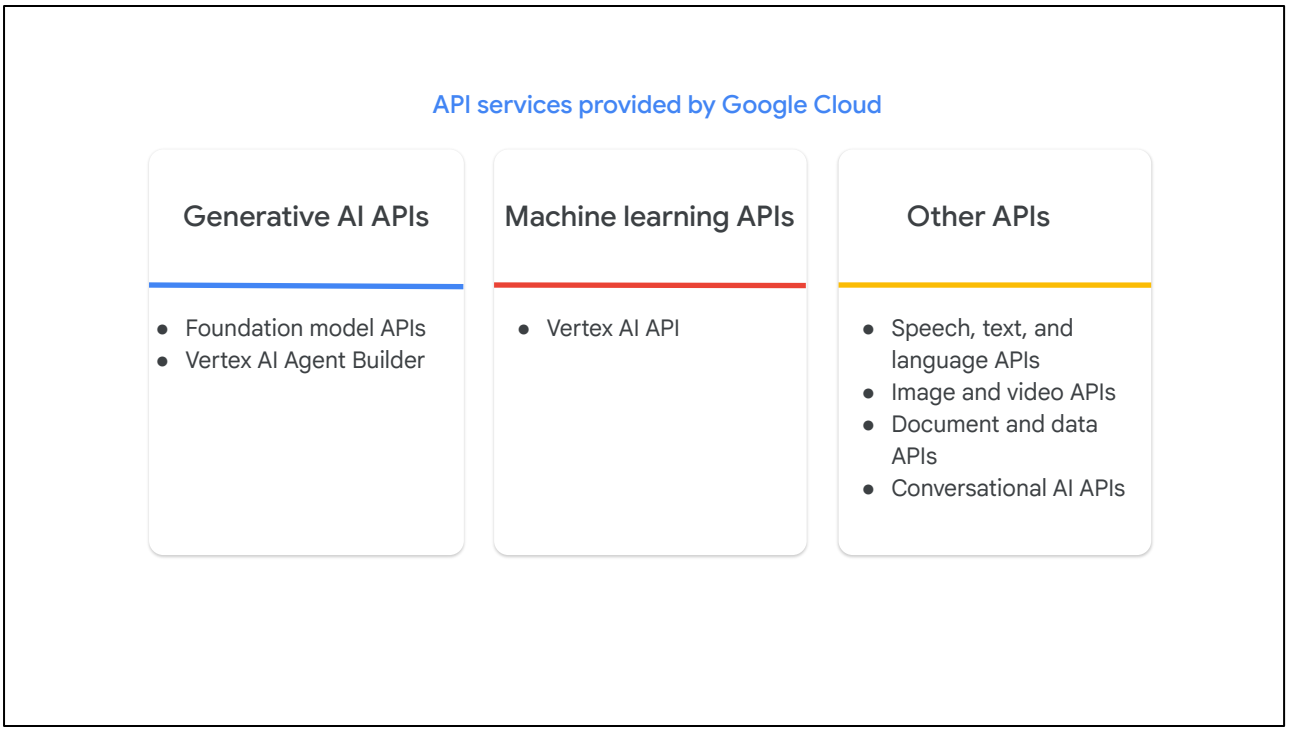


Look at a simple example. The code uses the Google AI for Python SDK (Software Development Kit) to communicate with the **Gemini API**, following these steps:

1. **Authenticate** your session by passing the unique API key as a parameter to the `genai.configure()` function, granting you permission to use the API., using the statement genai.configure(api_key="YOUR_API_KEY").
2. **Specify the Gemini models** you want to process your request, like Gemini 2.5 Flash, using the statement model = genai.GenerativeModel('gemini-2.5-flash').
3. Make an **API call** to Google's servers, sending the prompt data. This string of text, passed to the `model.generate_content()` function, serves as the main input—the question or instruction—that you want the AI to respond to. Follow the statement: response = model.generate_content ("What are the three largest countries by area?").
4. Receive the model's generated text back as a **response**, by using the statement print(response.text).



It's remarkable, isn't it? You don’t need to train your own large language models. Instead, you can access and utilize the pre-trained AI models directly through API calls in the same way as a function call.



So, what are the API services provided by Google Cloud?

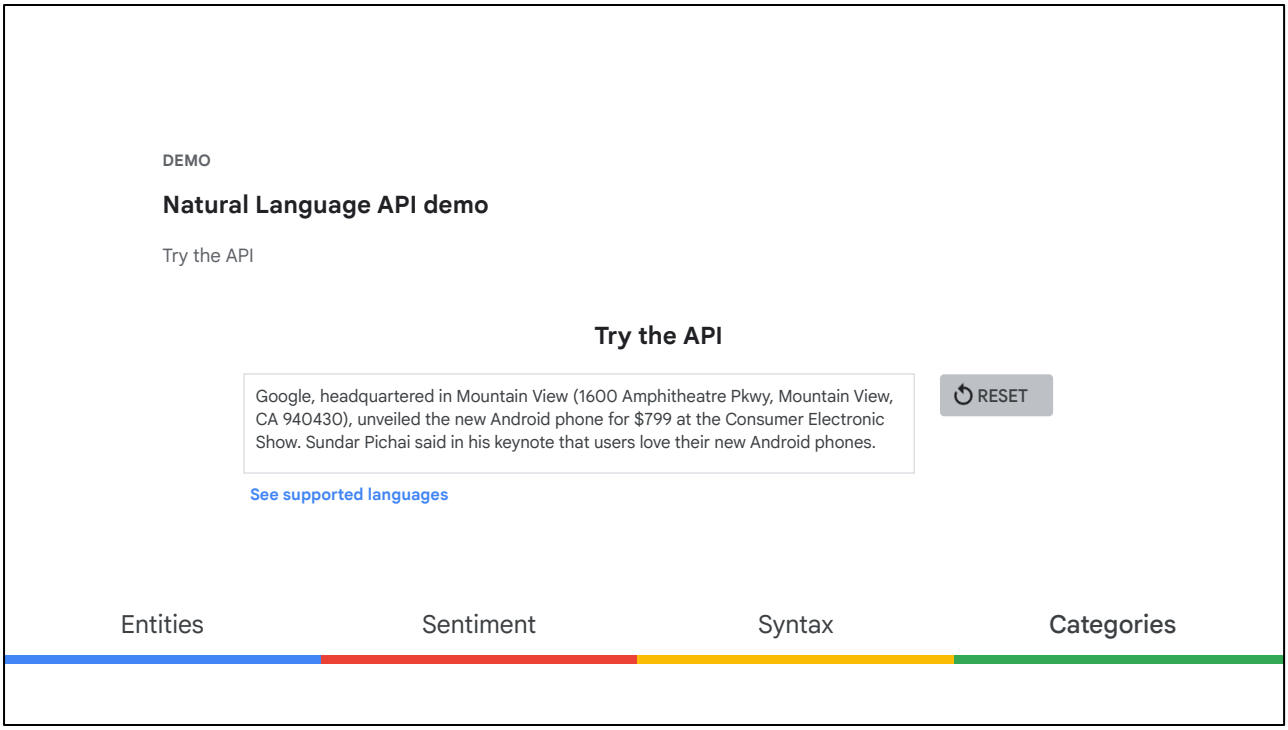
Let's explore a short list:

- **Generative AI APIs** include foundation model APIs, such as the multimodal Gemini APIs, which can be leveraged to directly create content. Additionally, Vertex AI Agent Builder provides a comprehensive suite of features for discovering, building, and deploying AI agents.
- **Machine Learning APIs**, like the Vertex AI API, can be used to train, monitor, and tune an ML model with minimal ML expertise and effort.
- **Other APIs** include speech, image, document, and conversation APIs. This list is constantly evolving with technological advancements. Many of these can be replaced by the Gemini APIs, which are considered multitask and multimodal.

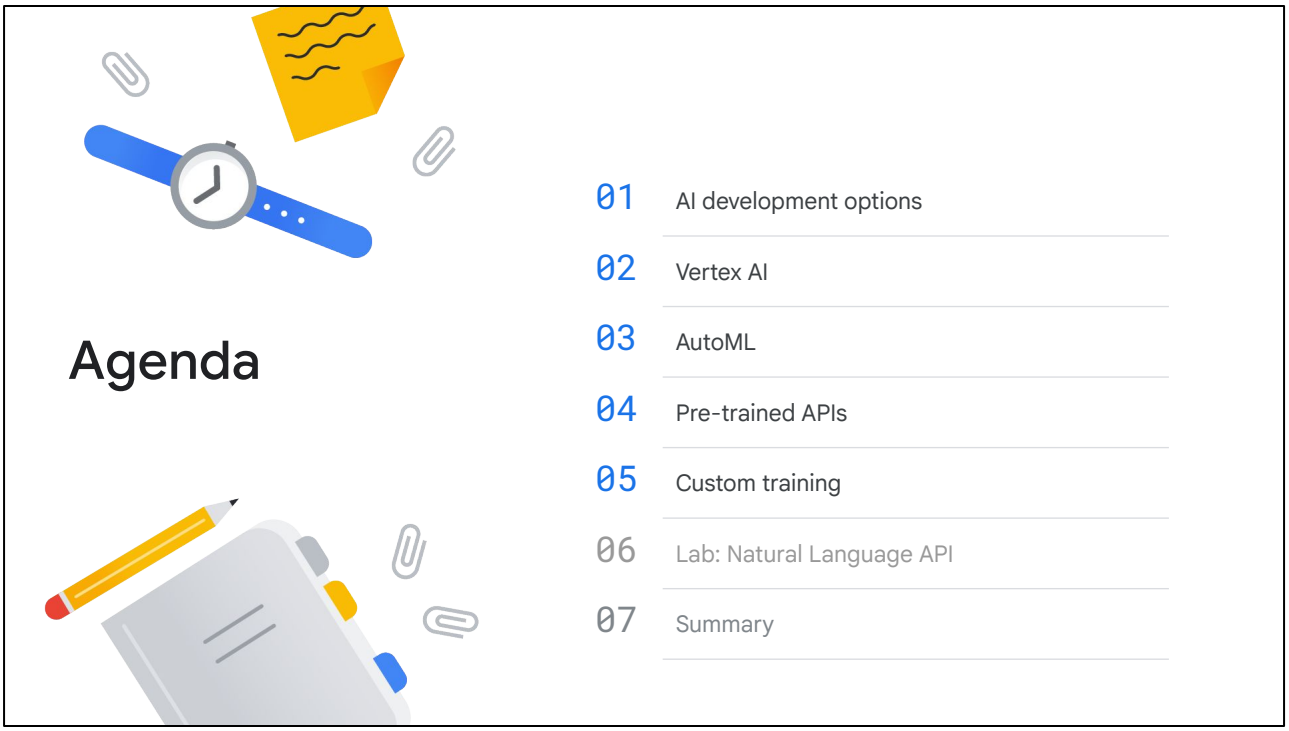
Natural Language API

https://cloud.google.com/natural-language#demo

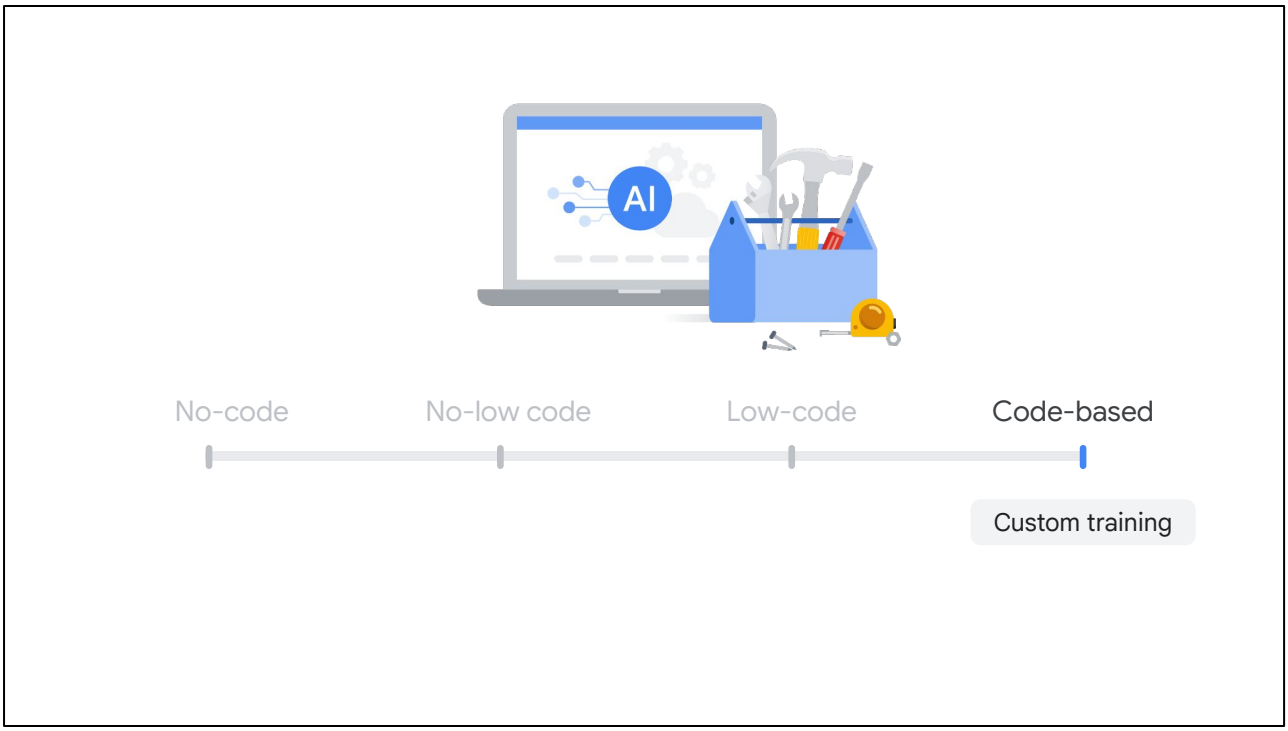
Ever wondered what magic lies behind understanding human language? Discover the Natural Language API, a powerful tool that allows you to analyze text directly in your browser.



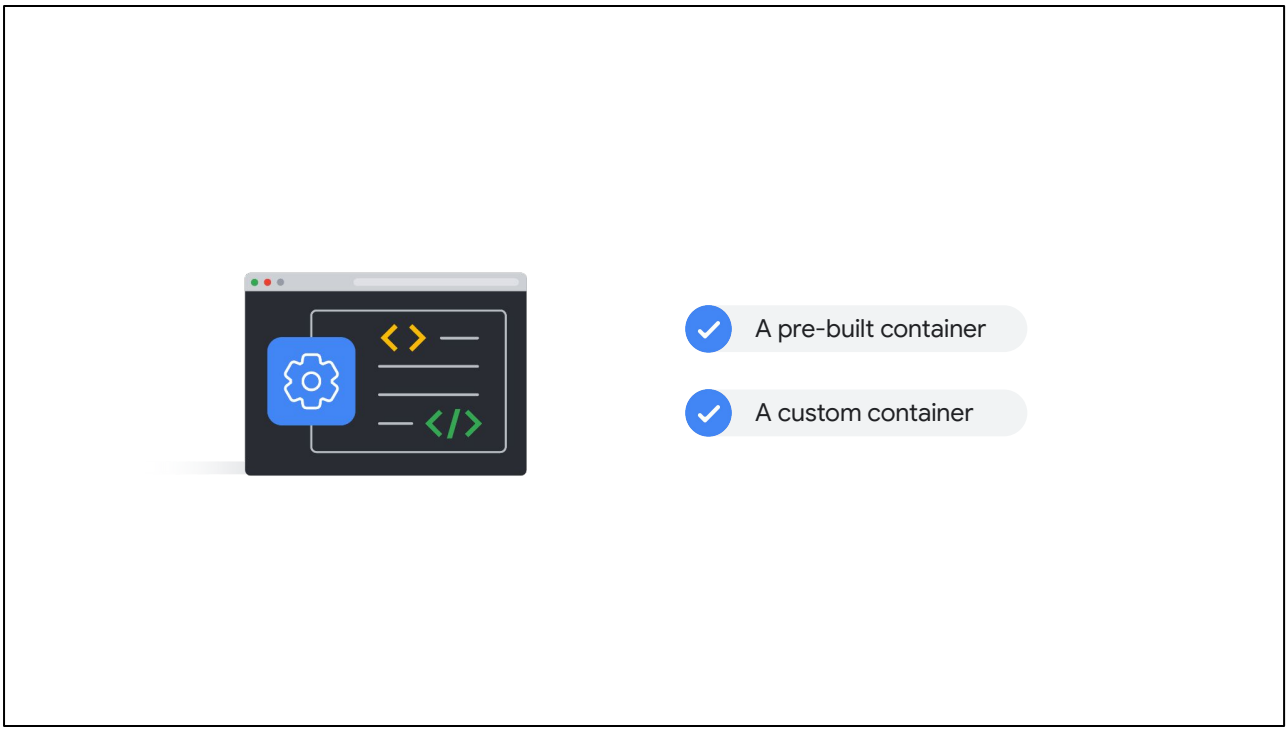
Uncover hidden insights by identifying entities, sentiment, syntax, and categories, transforming raw text into meaningful data.



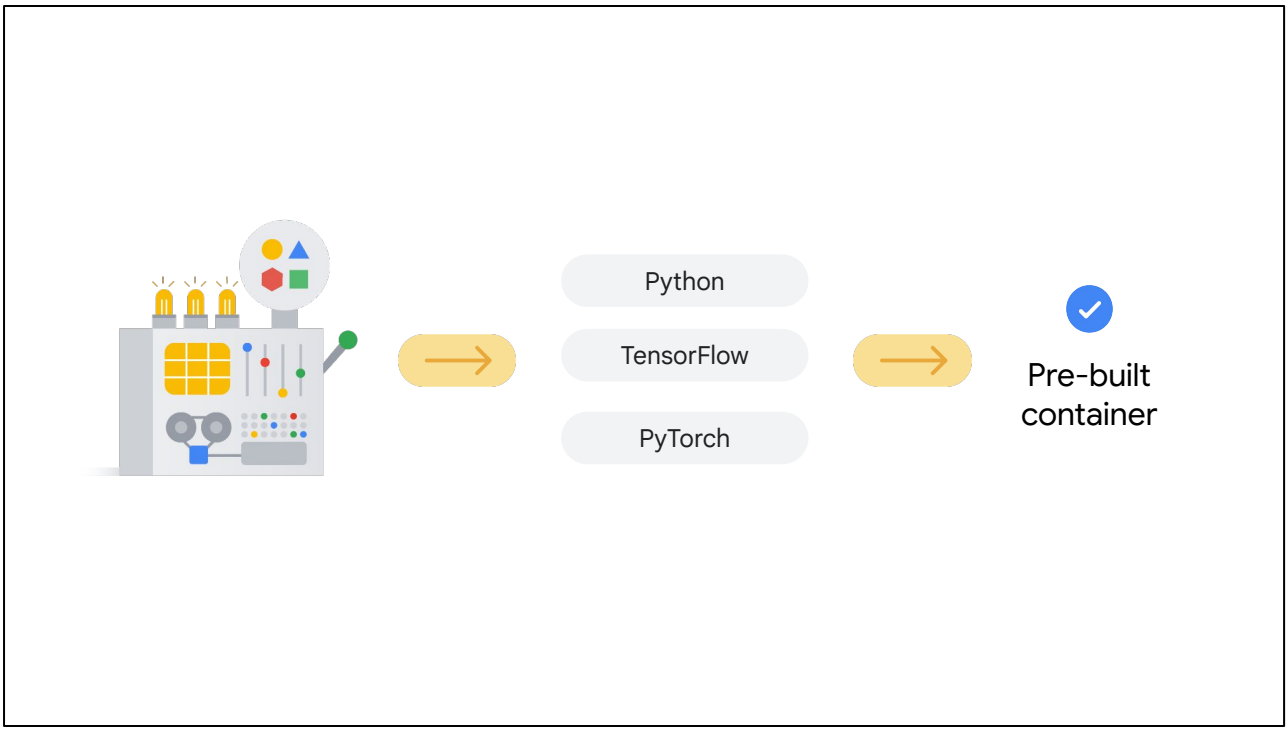
In previous lessons, you learned about no-code solutions like AutoML and low-code solutions like pre-trained APIs. Now, let's explore a code-based solution: custom training, a do-it-yourself approach to building an ML model.



While AutoML UI offers significant convenience and pre-trained APIs eliminate the need for training data, you might require custom training if your unique needs extend beyond AutoML's automated capabilities. This is when complete control and flexibility over the model architecture, frameworks, and training logic become essential.

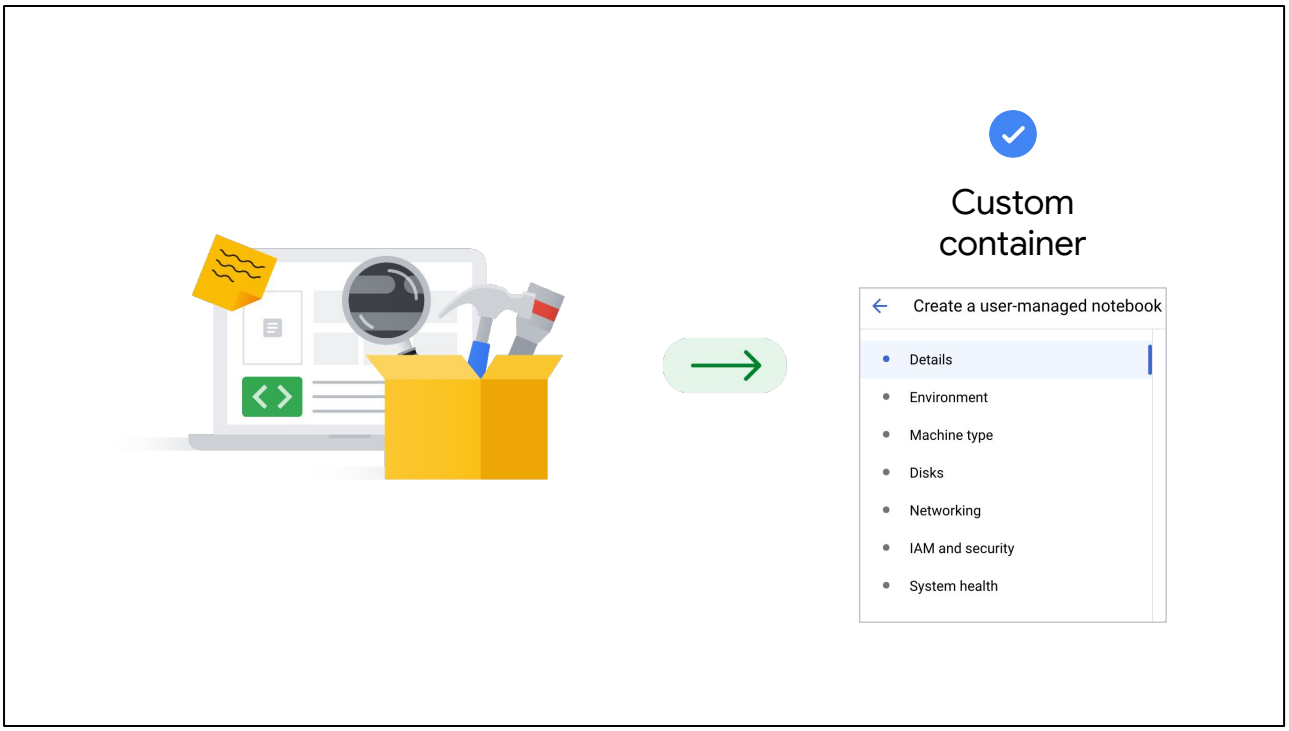


Before any coding begins, you must determine what environment you want your ML training code to use. There are two options: a **pre-built container** or a **custom container**.



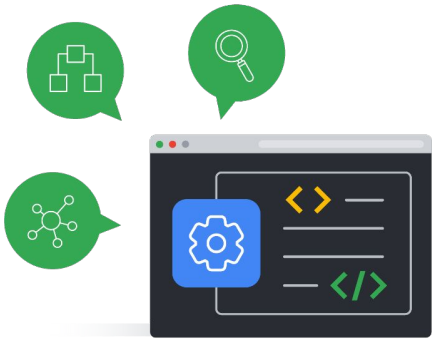
A pre-built container is like a furnished kitchen with cabinets, appliances, and cookware.

So, if your ML training needs a platform like Python, TensorFlow, and PyTorch and you're not particular about the underlying infrastructure to run on, or to use our kitchen analogy, which oven or knife you use, a pre-built container is probably your best choice.

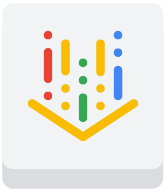


A custom container, alternatively, is like an empty room.

You define the exact appliances and tools that you prefer to cook with. That means you must determine the details like the environment, machine type, and disks when creating the custom container.



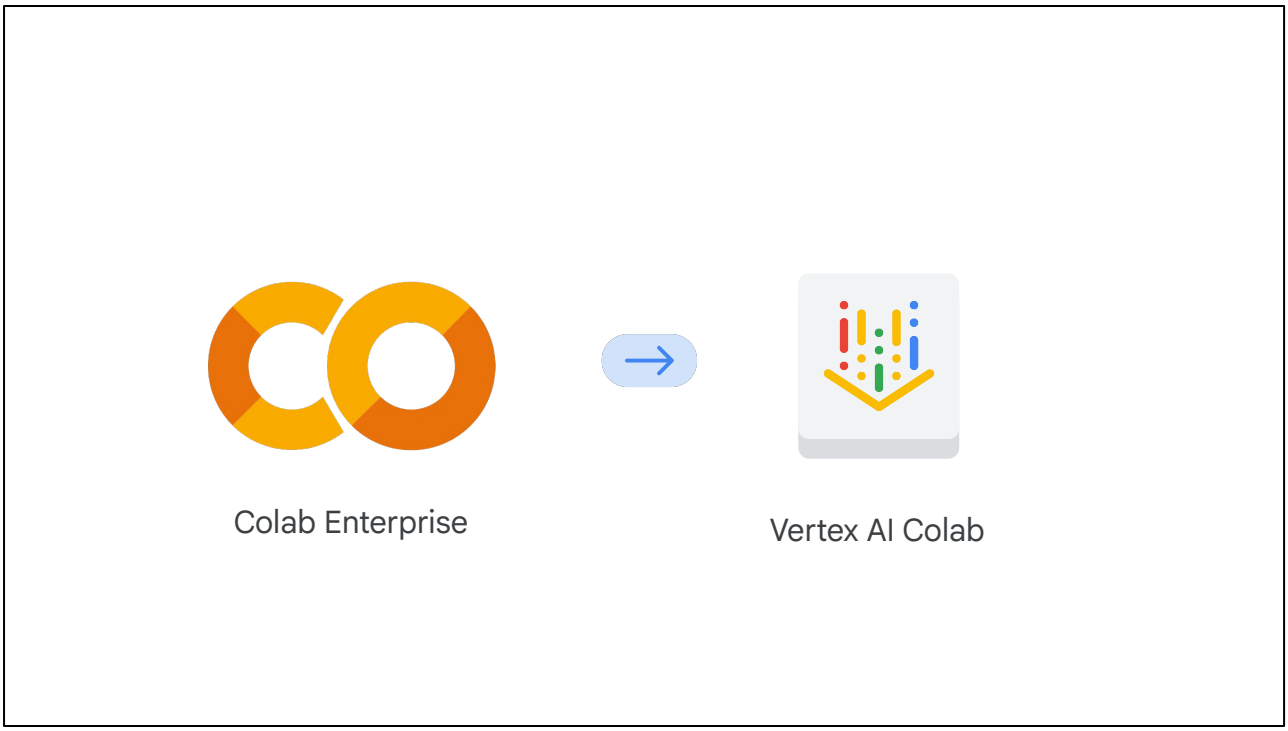
Single development environment



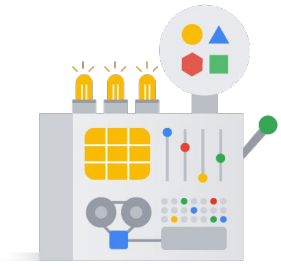
Vertex AI Workbench

In terms of the tools to code your ML model, you can use **Vertex AI Workbench**.

You can think of Vertex AI Workbench as a Jupyter notebook deployed in a **single development environment** that supports the entire data science workflow, from exploring to training and then deploying a machine learning model.



You can also use Colab Enterprise, which was integrated into Vertex AI Platform in 2023 so data scientists could code in a familiar environment.



ML library

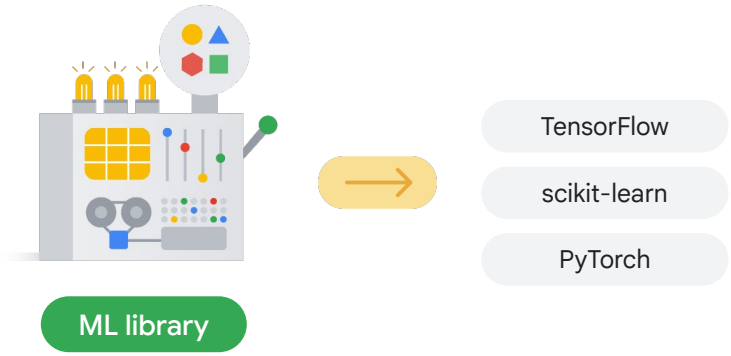
- A collection of pre-written code
- Tools to save time and effort

After you decide the working environment, the next step is to start writing code.

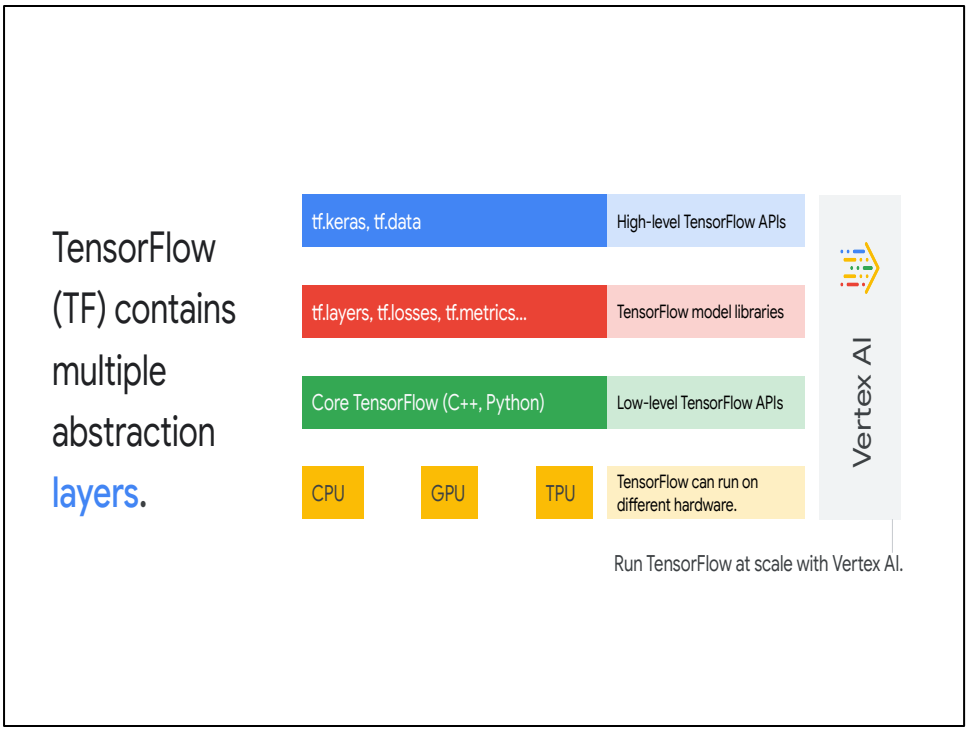
These days, you don't have to code from scratch. Instead, you can leverage ML libraries.

An ML library is a collection of pre-written code that can be used to perform machine learning tasks.

These libraries can save developers time and effort by providing them with the tools they need to build machine learning models without having to write everything from the beginning.



As a data scientist, you might already be familiar with popular ML libraries like TensorFlow, scikit-learn, and PyTorch. They are open source and widely used by a large community of users and developers.



Let's explore TensorFlow, an end-to-end open platform for machine learning supported by Google.

Tensorflow contains multiple abstraction layers. You use TensorFlow APIs to develop and train ML models. The TensorFlow APIs are arranged hierarchically, with the high-level APIs built on the low-level APIs.

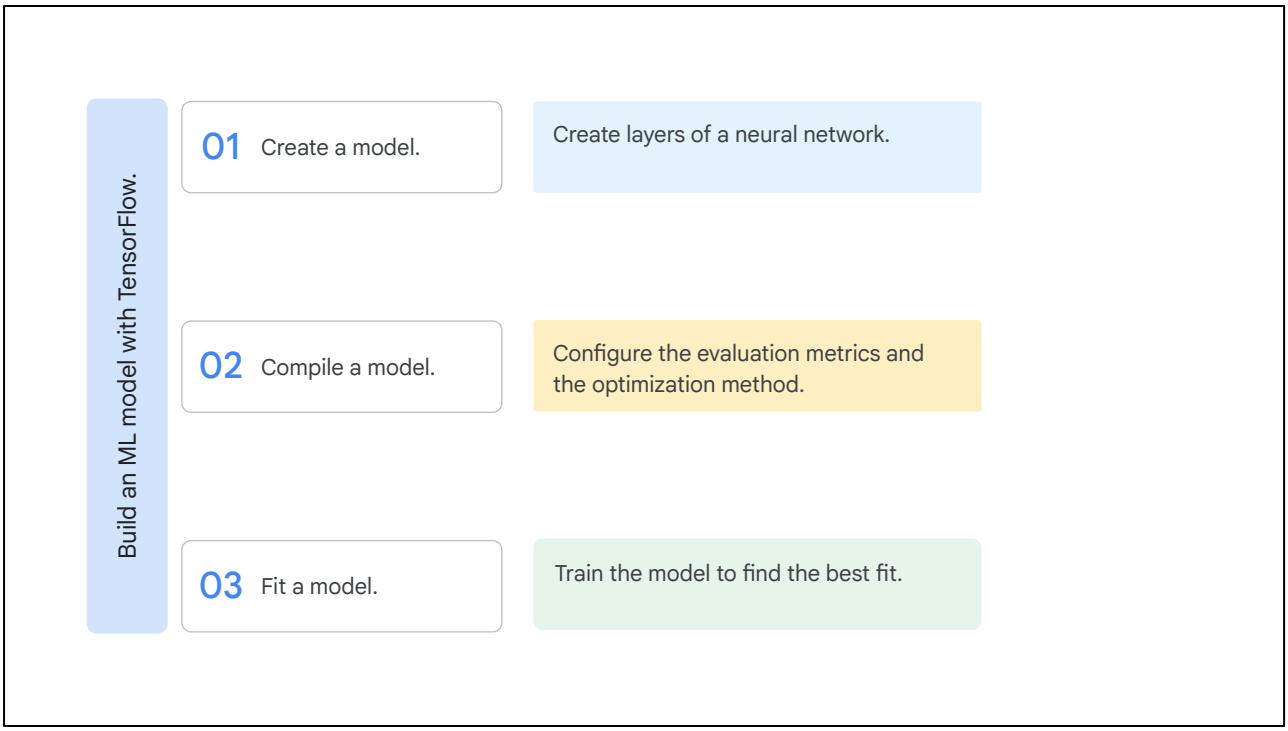
The lowest layer is hardware. TensorFlow can run on different hardware platforms including CPU, GPU, and TPU.

The next layer is the low-level TensorFlow APIs, where you can write your own operations in C++ and call the core, basic, and numeric processing functions written in Python.

The third layer is the TensorFlow model libraries, which provide the building blocks, such as neural network layers and evaluation metrics to create a custom ML model.

The high-level TensorFlow APIs like Keras sit on top of this hierarchy. They hide the ML building details and automatically deploy the training. They can be your most used APIs.

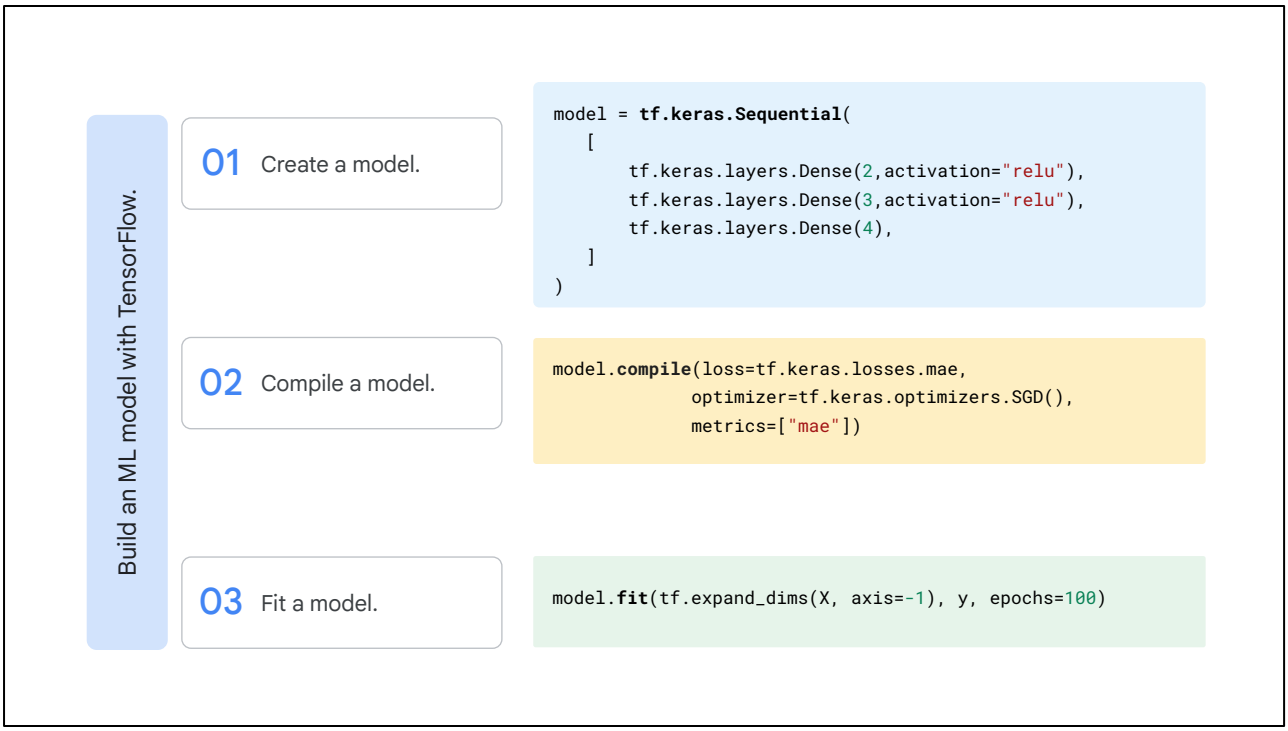
Note that Vertex AI fully hosts TensorFlow from low-level to high-level APIs. Regardless of which abstraction level you are writing your TensorFlow code at, Vertex AI gives you a managed service.



Now let's look at an example of using tf.keras, a high-level TensorFlow library commonly used, to build a simple regression model.

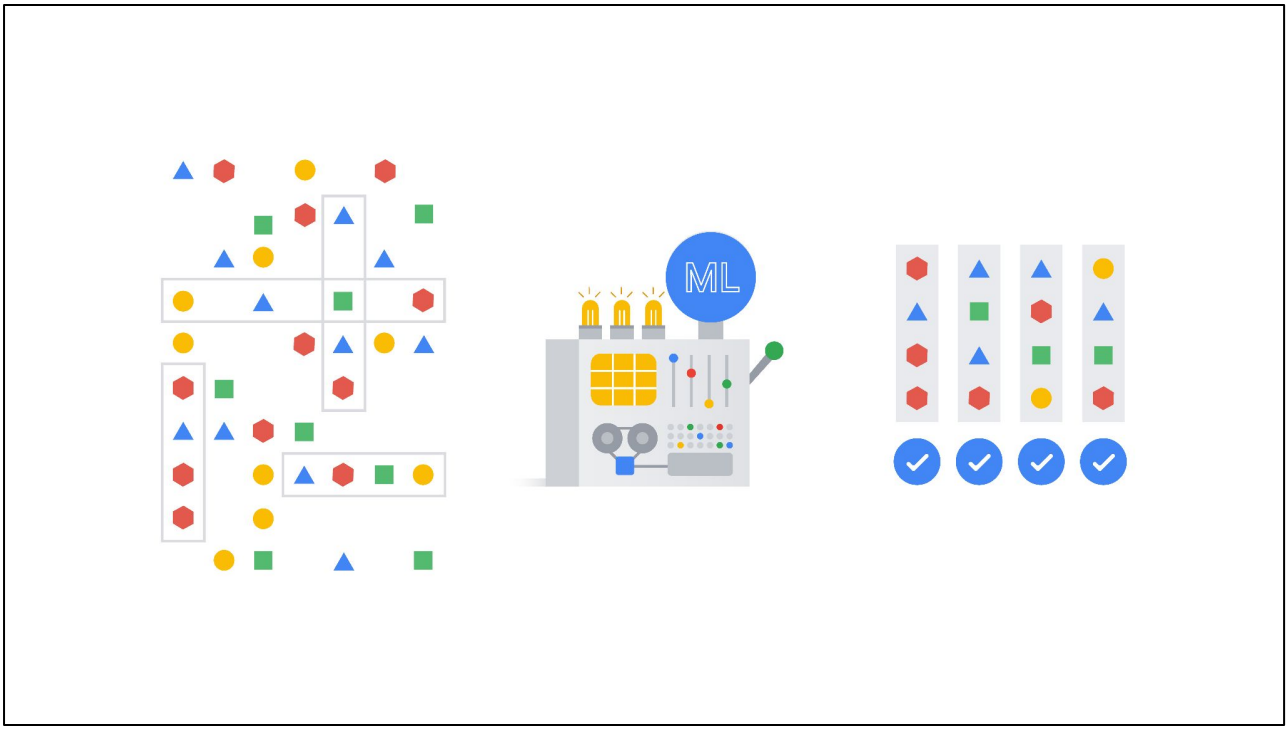
Typically, it takes three fundamental steps:

- In step one, you create a model, where you piece together the layers of a neural network.
- In step two you compile the model, where you specify hyperparameters such as performance evaluation and model optimization.
- Finally, you train your model to find the best fit.



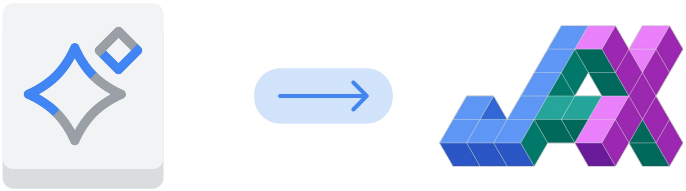
Assume you already imported necessary packages like TensorFlow and uploaded the data.

- The first step is to create a model by using tf.keras.Sequential. To demonstrate, you can define your model as a three-layer neural network. You'll explore more details about neural networks such as activation functions in the next module.
- The next step is to compile the model by specifying how you want to train it by using the method **compile**. For instance, you can decide how to measure the performance by specifying a loss function. You can also optimize the training by pointing to an optimizer.
- The last step is to train the model by using the method **fit**. For instance, you can define the input, the training data, and the output, the predicted results. You can also decide how many iterations you want to train the model by specifying the numbers of epochs.



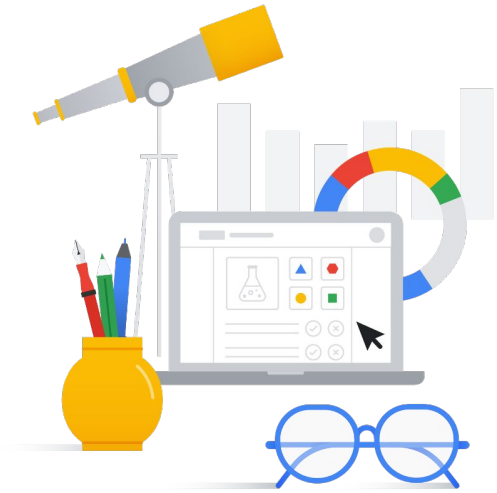
After you train the model and are satisfied with the performance, you can then deploy the model and make predictions.

JAX: a new framework for ML



Apart from Tensorflow, Google is consistently introducing new frameworks.

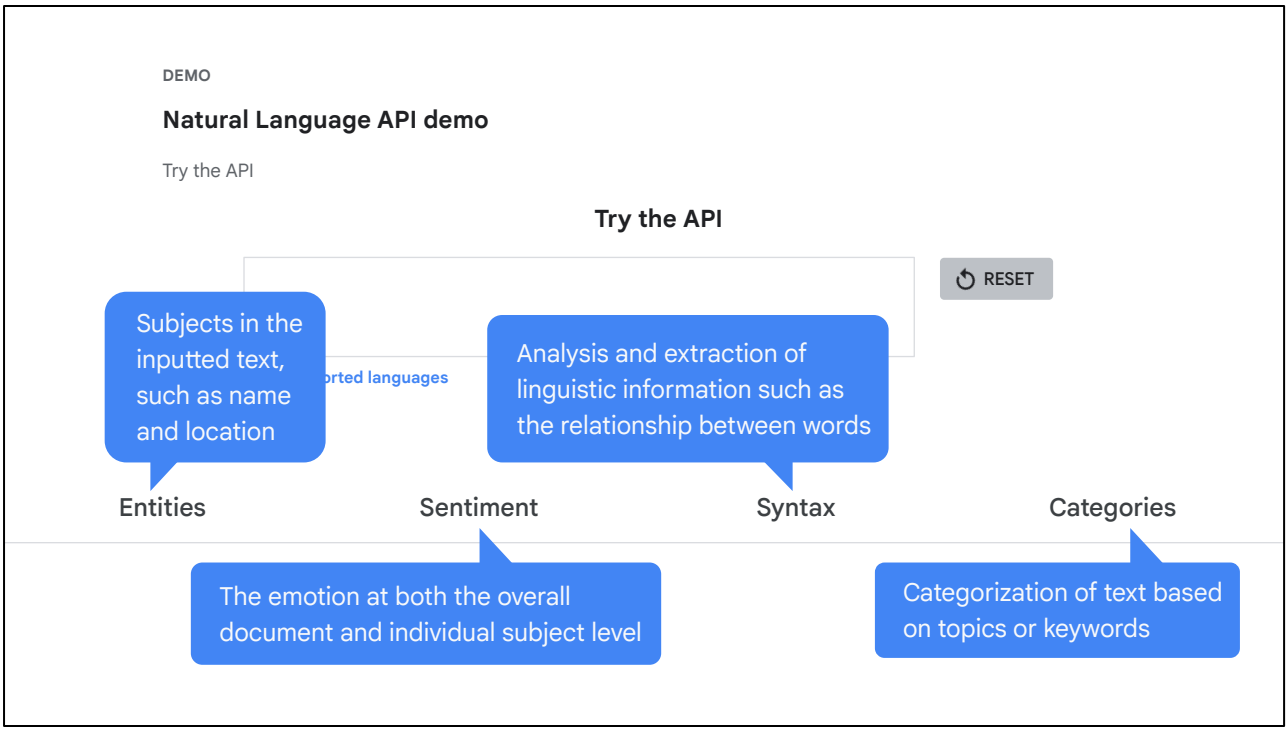
One of the most promising frameworks is JAX. JAX is a high-performance numerical computation library that is highly flexible and easy to use. It offers new possibilities for both research and production environments.



Hands-on lab:

Entity and Sentiment Analysis with the Natural Language API

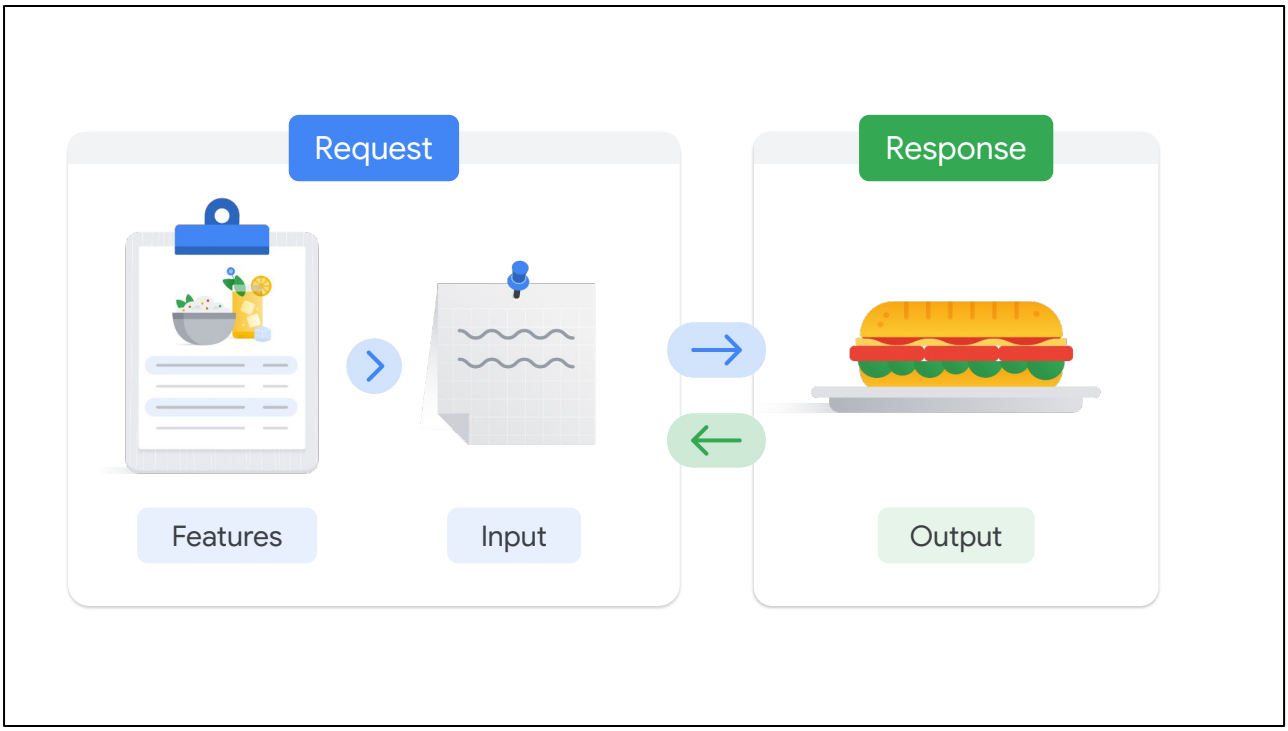
Now it's time for some hands-on practice. In the following lab, you'll use the Natural Language API to analyze text. Specifically, you'll identify entities and analyze sentiment with code.



Before you begin, let's briefly review the main features of the Natural Language API you learned in the previous lesson:

- You can identify **entities**, which are subjects in the inputted text, such as **Google** as a company name and **Mountain View** as a location.
- You can identify the **sentiment**, which indicates emotion at both the overall document and individual subject level.
- You can analyze **syntax** and extract linguistic information such as the relationship between words.
- And you can also classify the text to categories based on topics or keywords, similar to assigning a tag to a piece of text.

You perform all of this analysis through a UI, which is a quick and efficient way to demonstrate and test these features.



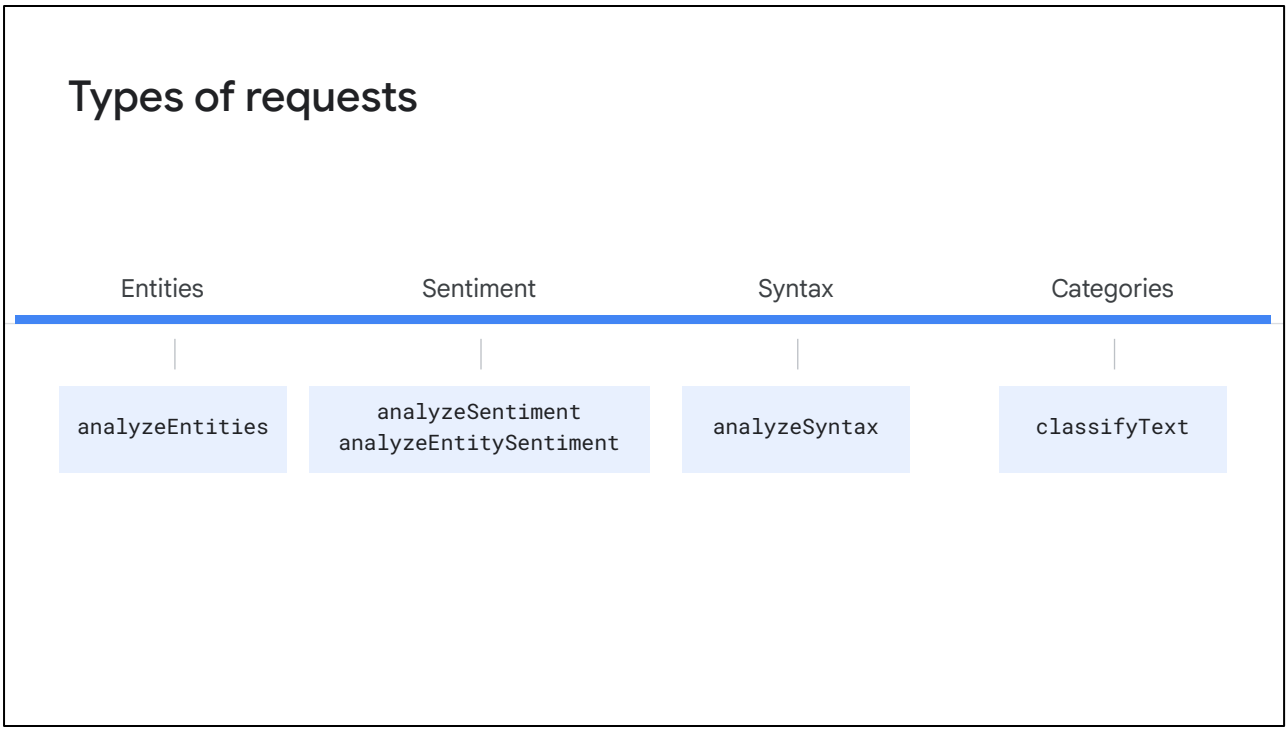
However, if you want to incorporate these features in production, you must embed the APIs into code.

Using APIs in your code is similar to ordering a sandwich at a deli. You order from the menu and get your food without worrying about how it was made in the kitchen. The same concept applies to using APIs. You only need to know three things: the features (the menu), the input (the order), and the output (the sandwich).

Like a menu, features are the types of requests that you can make to the Natural Language API.

Like a food order, the input is how how you construct the requests.

Then, the sandwich you receive after you place the order is the response (or output). With this, you can determine next steps.



So, what are different types of requests that you can make?

The Natural Language API provides several methods for performing analysis and annotation on your text.

You'll practice with most of them in the lab.

- For entity analysis, you can use the **analyzeEntities** method.
- The sentiment analysis is performed through the **analyzeSentiment** method at the entire text level, and **analyzeEntitySentiment** at the individual entity and subject level.
- The syntactic analysis is performed with the **analyzeSyntax** method.
- And the content classification is performed by using the **classifyText** method.

Construct the request: request.json

```
{
   "document":{
   "type":"PLAIN_TEXT",
   "language":"EN",
   "content":"'Lawrence of Arabia' is a highly rated film biography about British
  Lieutenant T.E. Lawrence. Peter O'Toole plays Lawrence in the film."
   },
   "encodingType":"UTF8"
}
```

Now, how do you construct those requests?

The Natural Language API is a REST API and consists of JSON requests and responses.

A simple JSON request for entity analysis looks like the code shown here, where you define:

1. The type of the document, for example, plain text.
2. The language, like **EN,** which stands for English.
3. The content, which can be the text itself or the file location in Cloud Storage, and finally
4. The encoding type like UTF8.

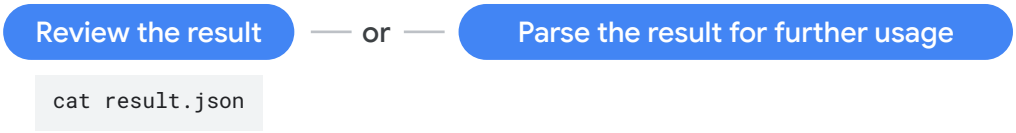
Call the APIs

```
curl "https://language.googleapis.com/v1/documents:analyzeEntities?key=${API_KEY}" \
  -s -X POST -H "Content-Type: application/json" --data-binary @request.json >
result.json
```

After you construct the request, you need to call the API. Just like, after you decide what you want at a deli, you need to place the order with the counter person. Here is an example to call the API with cURL. cURL stands for client URL and is a command-line tool to transfer data between client and server. You can also use other programming languages such as Python and Java SDKs to call the APIs. Typically, the vendors of the product and services you are using define the APIs and provide the SDKs in different languages for you to choose.

In this example, you call the Natural Language API feature **analyzeEntites**, pass the request.json file that you just constructed, and save the response to the **result.json** file.

Receive responses




```
cat result.json
```

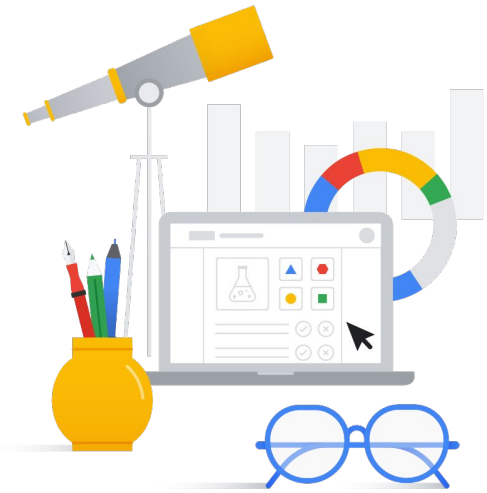
Finally, how should you handle the responses? You can review the result by using a command like **cat result.json** or parse it for further usage.

Tasks

01 Extract entities.

02 Analyze sentiment.

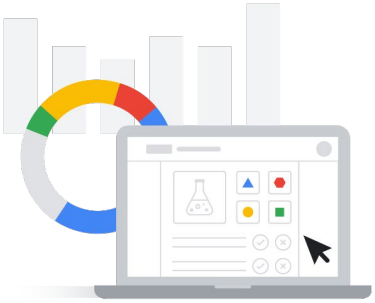
03 Analyze syntax.



Equipped with the technical details, in this lab, you'll use the Natural language API to:

- Extract entities.
- Analyze sentiment.
- And analyze syntax.

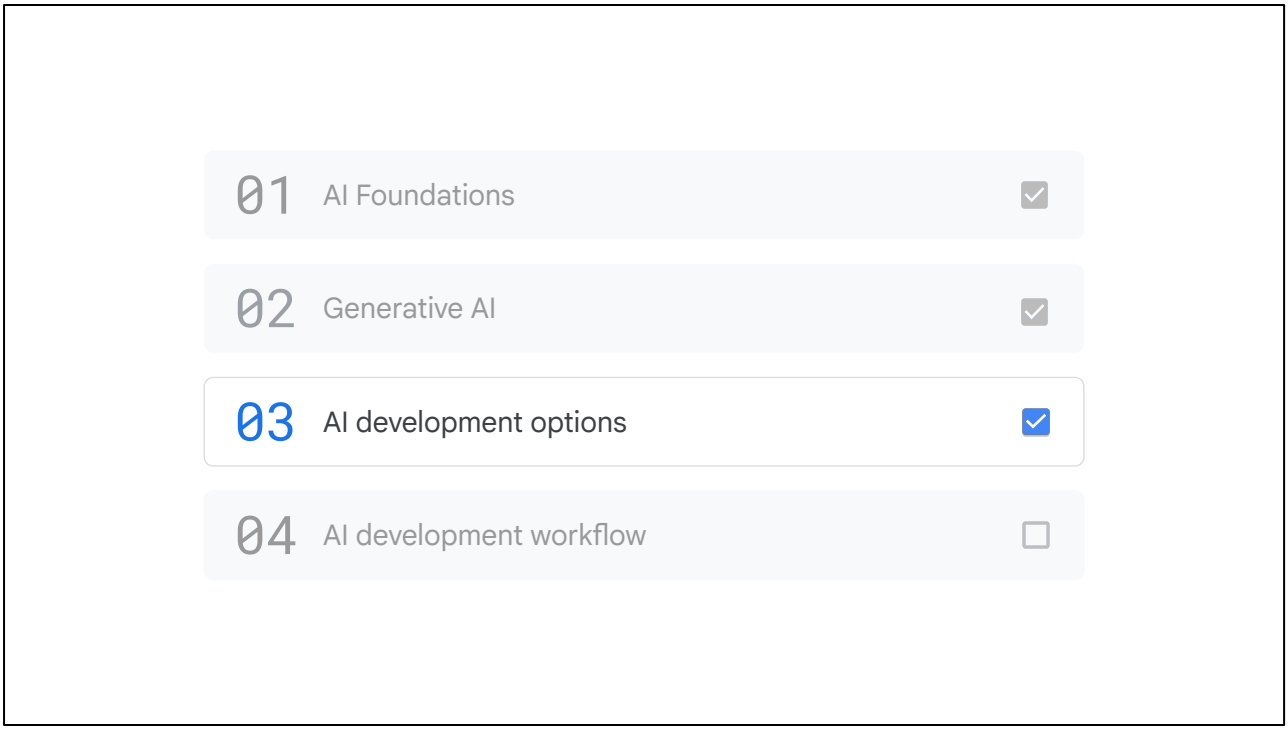
Objectives



- Create a Natural Language API request, and call the API with cURL.
- Extract entities, and run sentiment analysis on text.
- Perform linguistic analysis on text.
- Create a Natural Language API request in a different language.

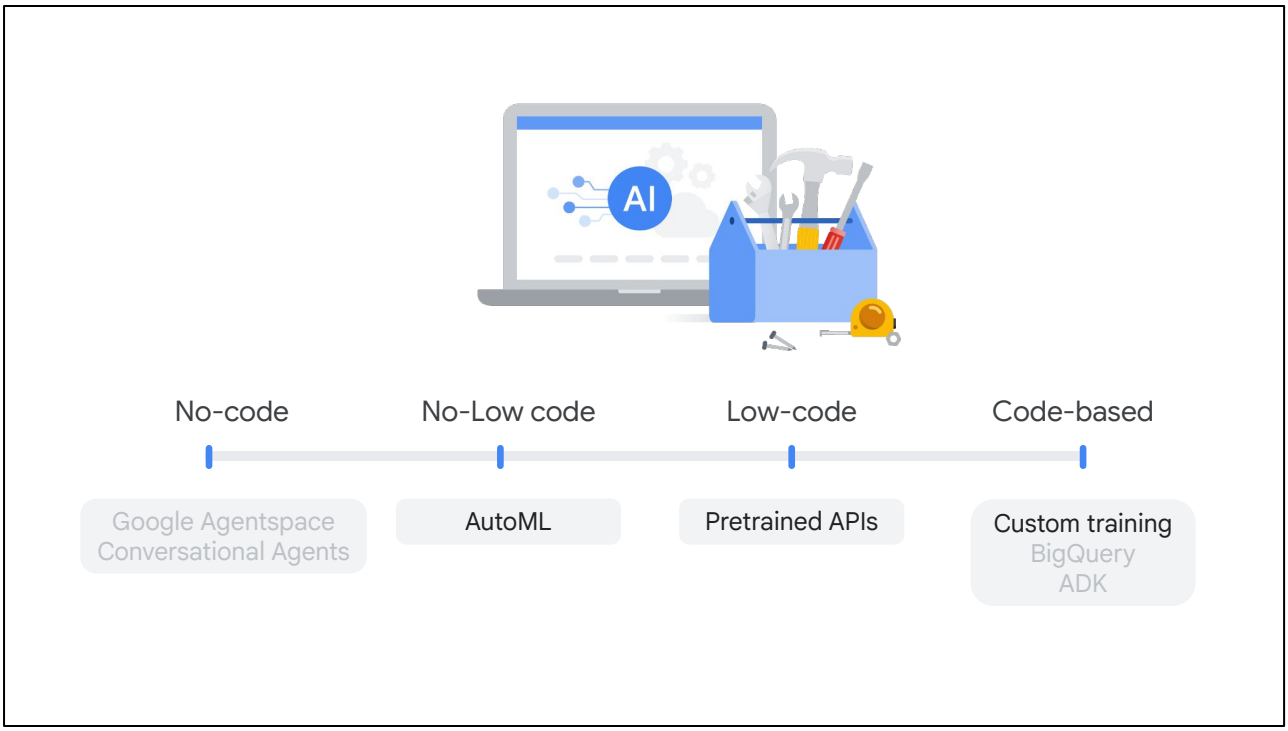
By completing the lab, you'll get practice:

- Creating a Natural Language API request and calling the API with cURL.
- Extracting entities and running sentiment analysis on text.
- Performing linguistic analysis on text.
- And creating a Natural Language API request in a different language.



Back to the original question for this module: What options do you have for building an AI project or an ML model? Do you have the answer now?

If not, let's take a quick look.



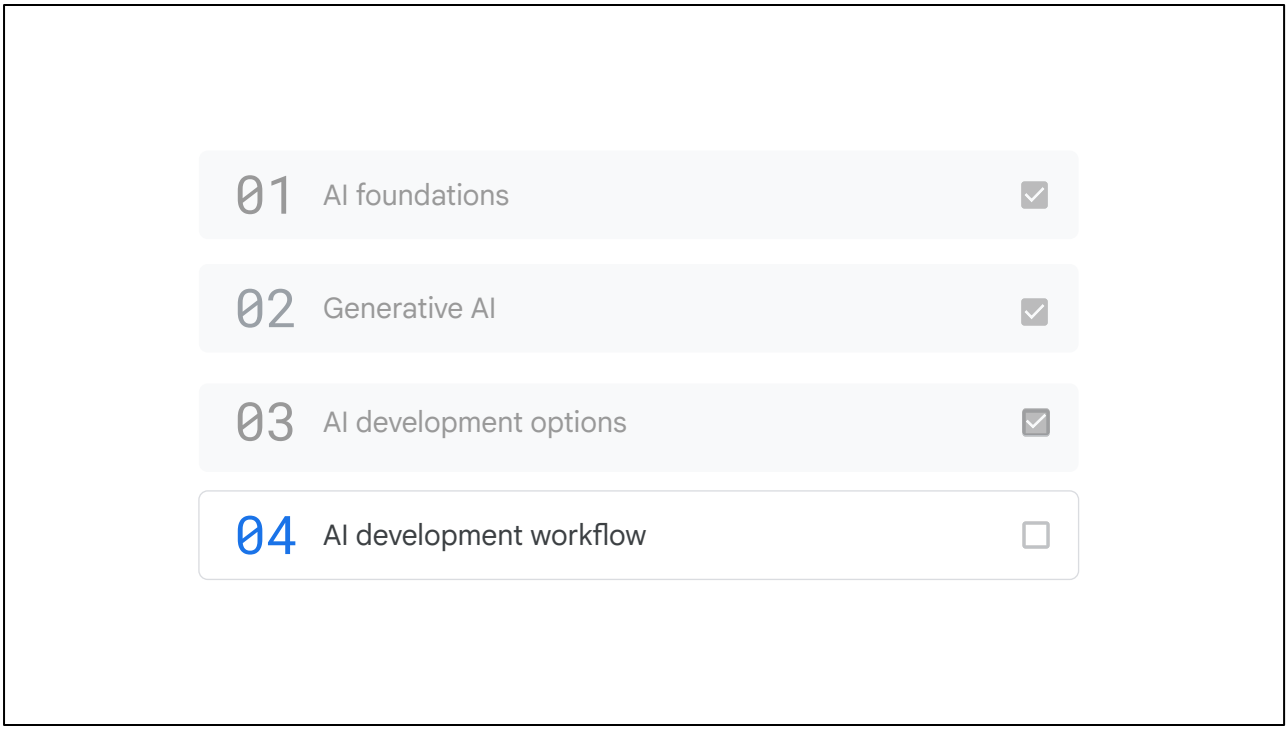
Google Cloud offers a wealth of options to suit your needs, whether you're a business user, data scientist, developer, or ML engineer.

These range from no-code, out-of-the-box solutions to low-code tools and even code-based DIY approaches.

You've been introduced to Vertex AI, Google's unified AI platform, which serves as your ultimate workspace for exploring these options and building end-to-end machine learning projects.

You've specifically focused on:

- **AutoML:** A fantastic low- or no-code tool within Vertex AI that automates ML development from data preparation to model training and serving, all using your own data.
- **Pre-trained APIs:** Ready-made solutions that leverage powerful pre-trained machine learning models, completely eliminating the need for any training data.
- **Custom training:** This empowers you to manually code ML projects using versatile tools such as Python, JAX, and Vertex AI Workbench.



Given all these exciting options, how do you build an ML model step-by-step on Vertex AI? Let’s uncover the answer in the next module.